МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ ГОМЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ П. О. СУХОГО

Факультет автоматизированных и информационных систем

Кафедра «Информационные технологии»

Специальность 6-05-0611-01 «Информационные системы и технологии»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
по дисциплине «Программирование сетевых приложений»

на тему: «Клиент-серверная система резервного копирования файлов с автоматической синхронизацией и ведением журнала изменений»

Исполнитель: студент гр. ИТД-41
Лаврентьев А.В
Руководитель: доцент
Комраков В.В

Дата проверки: _____
Дата допуска к защите: _____
Дата защиты: _____
Оценка работы: _____

Подписи членов комиссии

по защите курсовой работы: _____

Гомель 2026

СОДЕРЖАНИЕ

Введение	4
1 Выбор и обоснование технологического стека системы	6
1.1 Характеристика предметной области и требования к системе резервного копирования	6
1.2 Платформа *.NET* и средства клиент-серверного взаимодействия	8
1.3 Средства пользовательского интерфейса и хранения данных	10
2 Архитектура и алгоритмы системы резервного копирования	14
2.1 Функциональная структура и взаимодействие компонентов системы	14
2.2 Организация сетевого обмена и модель общего хранения данных	15
2.3 Механизмы синхронизации, версионирования, разрешения конфликтов и восстановления файлов	18
3 Программная реализация клиент-серверной системы	22
3.1 Структура решения и общие программные компоненты	22
3.2 Реализация клиентского приложения и шлюза синхронизации	24
3.3 Реализация сервисов метаданных и файлового хранилища	28
4 Тестирование и верификация программного комплекса	30
4.1 Методика и условия тестирования программного комплекса	30
4.2 Результаты автоматизированного и интеграционного тестирования	31
4.3 Проверка функциональности и отказоустойчивости системы	33
Заключение	37
Список использованных источников	38
Приложение А – Архитектурная схема взаимодействия компонентов	39
Приложение Б – Текст программы	40
Приложение В – Результаты автоматизированного тестирования	72
Приложение Г – Демонстрация работы приложения	74
Приложение Д – Руководство пользователя	77
Приложение Е – Руководство программиста	79
Приложение Ж – Руководство системного программиста	81

ВВЕДЕНИЕ

Современные информационные системы и персональные рабочие станции ежедневно создают и изменяют значительные объёмы файловых данных. Документы, исходные тексты программ, результаты расчётов, конфигурационные файлы и другие материалы могут многократно изменяться в ходе работы. Случайное удаление, повреждение файловой системы, отказ накопителя либо некорректное редактирование способны привести к потере актуальной информации и необходимости повторного выполнения уже выполненной работы. Поэтому резервное копирование является одним из основных средств обеспечения сохранности пользовательских данных и устойчивости программной среды.

Ручное создание резервных копий зависит от действий пользователя и не обеспечивает гарантированного сохранения каждого значимого изменения. При активной работе возникает необходимость хранить не только последнее состояние файла, но и последовательность его предыдущих версий, чтобы при ошибке можно было восстановить требуемое состояние. Дополнительную ценность представляет журнал изменений, позволяющий определить время и характер операций с файлами, включая создание, изменение, переименование, удаление и восстановление. Таким образом, практическая система резервного копирования должна объединять автоматическое наблюдение за локальным каталогом, сетевую передачу данных, хранение истории версий и средства восстановления.

Важным требованием к подобной системе является поддержка работы нескольких клиентских рабочих станций с общим набором файлов. Пользователи могут подключаться к одному общему хранилищу, получать актуальные версии файлов и передавать собственные изменения. При этом система должна обеспечивать согласованность между локальными каталогами разных клиентов и серверным состоянием, сохранять сведения об авторе каждой операции и не допускать потери данных при одновременном изменении одного и того же файла на нескольких рабочих станциях. Для разрешения подобных ситуаций может использоваться создание конфликтной копии, позволяющей сохранить расходящуюся локальную версию без перезаписи актуального серверного файла.

Для решения указанных задач применяется клиент-серверная архитектура. Клиентская часть располагается на рабочей станции пользователя, предоставляет веб-интерфейс, отслеживает выбранный каталог и формирует сведения об обнаруженных изменениях. Серверная часть разделена на несколько функциональных компонентов: шлюз сетевого взаимодействия, сервис метаданных и сервис файлового хранилища. Такое разделение позволяет разграни-

чить обработку сетевых соединений, работу с логической структурой данных и физическое хранение содержимого файлов.

Сетевой обмен между клиентами и серверными компонентами выполняется поверх *TCP* с использованием собственного прикладного протокола. Управляющая информация передаётся в структурированном виде, а содержимое файлов – в виде бинарного потока без дополнительного кодирования. Протокол должен обеспечивать корректное разделение сообщений в непрерывном *TCP*-потоке, поддержку файлов различного размера и однозначное связывание запросов с ответами. Для контроля целостности передаваемых и сохраняемых данных используется алгоритм *SHA*-256.

Автоматическая синхронизация строится на сочетании наблюдения за локальной файловой системой и периодического согласования состояния с сервером. Операционная система и прикладные программы могут формировать несколько событий файловой системы в рамках одного пользовательского действия, поэтому непосредственная передача каждого события способна привести к созданию избыточных версий. Перед синхронизацией необходимо учитывать фактическое состояние файла, его размер, контрольную сумму и текущую версию. Для общего набора серверное состояние периодически сравнивается с локальным индексом клиента, благодаря чему изменения, выполненные на одной рабочей станции, автоматически распространяются на остальные подключённые клиенты.

Отдельное значение имеет устойчивость системы к временной недоступности сети или серверного шлюза. При потере соединения клиентское приложение должно сохранять работоспособность интерфейса, не блокировать пользователя и фиксировать необходимость последующего согласования. После восстановления связи выполняется автоматическое переподключение и повторная синхронизация, в ходе которой накопившиеся локальные изменения передаются на сервер, а изменения других клиентов загружаются на рабочую станцию. Такой подход позволяет поддерживать согласованное состояние общего хранилища без постоянного ручного запуска синхронизации.

Метаданные о клиентах, наборах резервного копирования, участниках общего набора, файлах, версиях и журнале изменений хранятся в реляционной базе данных. При этом бинарное содержимое версий размещается отдельно в файловом хранилище. Сохранённые версии рассматриваются как неизменяемые: новое содержимое формирует очередную версию, а удаление исходного файла не уничтожает ранее сохранённые данные. Восстановление исторической версии не переписывает существующую историю, а создаёт новое актуальное состояние файла и фиксируется отдельной записью в журнале.

1 ВЫБОР И ОБОСНОВАНИЕ ТЕХНОЛОГИЧЕСКОГО СТЕКА СИСТЕМЫ

1.1 Характеристика предметной области и требования к системе резервного копирования

Резервное копирование предназначено для сохранения работоспособной копии данных, которая может быть использована после случайного удаления, повреждения исходных файлов, сбоя программного обеспечения или отказа накопителя. Для пользовательских рабочих станций особенно важна автоматизация этой задачи: при ручном копировании результат зависит от регулярности действий пользователя, а между двумя запусками резервного копирования могут возникнуть изменения, которые останутся несохранёнными.

В рекомендациях *NIST* по обеспечению устойчивости информационных систем отдельно отмечается целесообразность автоматизированного резервного копирования данных клиентских систем в централизованное расположение, а также необходимость документирования восстановительных операций. Данный подход соответствует задаче курсового проекта, в которой локальный клиент должен передавать изменённые файлы серверной части без постоянного участия пользователя, а сведения о совершённых действиях должны фиксироваться в журнале изменений *[1]*.

Для разрабатываемой системы недостаточно сохранять только последнюю копию файла. Пользователь может обнаружить ошибку спустя несколько изменений, поэтому требуется история версий, позволяющая выбрать более раннее состояние. Каждая успешно сохранённая редакция должна рассматриваться как самостоятельная неизменяемая версия. Удаление локального файла при этом не должно приводить к физическому уничтожению уже созданных резервных копий, иначе система не сможет выполнять восстановление после ошибочного удаления.

Важным элементом предметной области является синхронизация состояния локального каталога и серверного хранилища. Клиент должен фиксировать создание, изменение, переименование и удаление файлов, передавать соответствующую информацию серверу и сохранять новые версии только при фактическом изменении содержимого. Для общего набора к локальному отслеживанию добавляется получение актуального состояния с сервера, благодаря чему изменения одного участника автоматически распространяются на другие рабочие станции. При расхождении локальной и серверной редакций данные пользователя не должны теряться: расходящееся содержимое сохраняется как конфликтная копия. При временном отсутствии соединения изменения также не

должны теряться, а после восстановления связи требуется повторное согласование локального каталога с серверными метаданными.

Функциональные требования к программному комплексу формируются с учётом пользовательского сценария и особенностей сетевого взаимодействия. Пользователь выбирает локальный каталог и режим набора резервного копирования – личный либо общий, после чего приложение выполняет первичное согласование и переходит к автоматическому отслеживанию изменений. Для общего набора владелец получает код подключения, а другие зарегистрированные клиенты могут присоединиться к этому набору и работать с тем же серверным состоянием файлов.

Основные требования к разрабатываемой системе приведены в таблице 1.1.

Таблица 1.1 – Основные требования к системе резервного копирования

Категория	**Требование**	**Назначение**
Синхронизация	Автоматическое выявление локальных изменений и получение изменений других клиентов	Поддержание согласованного состояния общего набора
Версионирование	Сохранение последовательных неизменяемых версий	Возможность возврата к более раннему состоянию файла
Восстановление	Получение выбранной версии и безопасная замена локального файла	Возврат данных после ошибки, удаления или некорректного изменения
Журналирование	Фиксация операций и клиента, выполнившего изменение	Прослеживаемость действий участников и контроль состояния системы
Надёжность	Повторное подключение, согласование и сохранение конфликтных копий	Предотвращение потери данных при сетевых сбоях и расходящихся изменениях
Целостность	Проверка размера и *SHA*-256 при передаче и восстановлении	Обнаружение повреждения или неполной передачи файловых данных

Кроме функциональных требований, для программного комплекса существенны требования к надёжности и сопровождаемости. Сетевые операции должны выполняться без блокирования пользовательского интерфейса, в том числе при временной недоступности шлюза, а файлы большого объёма необходимо передавать потоково. Метаданные и физические версии должны оставаться согласованными даже при частичном отказе одного из серверных компонентов. Пользовательский интерфейс должен предоставлять понятные средства настройки каталога, создания личного или общего набора, подключения к общему набору, просмотра состояния синхронизации, истории файлов и журнала изменений.

Таким образом, предметная область определяет необходимость сочетания нескольких групп технологий: средств сетевого взаимодействия, асинхронного ввода-вывода, наблюдения за файловой системой, сериализации служебных сообщений, построения пользовательского интерфейса и надёжного хранения структурированных метаданных. Выбор конкретного технологического стека должен обеспечивать совместную реализацию этих требований в рамках единого программного комплекса.

1.2 Платформа *.NET* и средства клиент-серверного взаимодействия

В качестве основного языка программирования выбран *C#*, а в качестве программной платформы – *.NET* 9. Такое решение позволяет использовать единый язык и единые библиотеки для локального клиента, сетевого шлюза и серверных сервисов. *.NET* предоставляет управляемую среду выполнения, строгую систему типов, автоматическое управление памятью, развитую стандартную библиотеку и инструменты для построения сетевых, файловых и веб-приложений. В составе *.NET* 9 также используется язык *C#* 13, а сама версия платформы ориентирована, в частности, на повышение производительности и развитие средств серверной разработки *[2]*.

Для сетевого приложения существенным является наличие удобной модели асинхронного программирования. Передача файлов, ожидание входящих подключений, работа с базой данных и доступ к файловой системе относятся преимущественно к операциям ввода-вывода. Их синхронное выполнение приводило бы к простаиванию рабочих потоков и усложняло обслуживание нескольких клиентов. Модель *Task-based Asynchronous Pattern* и ключевые слова *async* и *await* позволяют освобождать поток на время ожидания внешней операции и продолжать выполнение после её завершения, что особенно важно для сетевых серверов и интерактивного клиента *[3]*.

Взаимодействие между клиентом и серверной частью построено поверх транспортного протокола *TCP*. *TCP* обеспечивает установление соединения и надёжную упорядоченную доставку потока байтов, но не определяет границы прикладных сообщений. Для работы с *TCP* в стандартной библиотеке *.NET* предоставлены классы *TcpClient* и *TcpListener* пространства имён *System.Net.Sockets*. *TcpClient* предназначен для установления клиентского соединения и получения *NetworkStream*, а *TcpListener* используется серверной стороной для прослушивания порта и приёма входящих подключений *[4]*.

Выбор стандартных сокетных средств вместо высокоуровневого *REST* или *gRPC* обусловлен характером учебной задачи и необходимостью явно реализовать сетевой обмен. Поскольку *TCP* представляет непрерывный поток, прикладной протокол должен самостоятельно задавать границы сообщений, размер полезной нагрузки и порядок чтения данных. Такое решение позволяет использовать один транспорт как для коротких управляющих сообщений, так и для потоковой передачи бинарного содержимого файлов.

Для автоматического обнаружения изменений в контролируемом каталоге применяется стандартный механизм *FileSystemWatcher*. Данный класс отслеживает уведомления файловой системы и формирует события при изменении файлов или каталогов. Это позволяет реагировать на создание, изменение, переименование и удаление без периодического полного сканирования каталога. При этом события наблюдателя должны рассматриваться как сигнал к проверке фактического состояния файла, поскольку одна пользовательская операция может породить несколько уведомлений *[5]*. Для общего набора событийное отслеживание локальных изменений дополняется периодическим получением актуального состояния файлов с сервера, поскольку *FileSystemWatcher* не может обнаруживать изменения, выполненные на других рабочих станциях.

Управляющая часть сетевых сообщений представляется в формате *JSON*. Для сериализации и десериализации используется встроенная библиотека *System.Text.Json*, входящая в стандартную платформу *.NET*. Она преобразует объекты *C#* в переносимое текстовое представление и обратно, работает с *UTF*-8 и ориентирована на высокую производительность и низкое количество выделений памяти. Использование стандартного сериализатора уменьшает количество сторонних зависимостей и обеспечивает единый формат служебных сообщений для всех компонентов системы *[6]*.

C# и *.NET* в данном проекте позволяют объединить сетевые сокеты, асинхронный ввод-вывод, работу с файловой системой, криптографические хеш-функции и сериализацию в одной технологической среде. Это упрощает совместимость клиентской и серверной частей, повторное использование общих контрактов и сопровождение программного комплекса. Для учебного проекта та-

кой подход также позволяет сосредоточиться на устройстве сетевого взаимодействия и алгоритмах синхронизации, не вводя дополнительные языки и среды выполнения.

1.3 Средства пользовательского интерфейса и хранения данных

Пользовательский интерфейс локального клиента реализуется как веб-интерфейс на основе *ASP.NET Core Blazor Web App*. При таком подходе пользователь работает с приложением в обычном браузере, а основная логика клиента продолжает выполняться в локальном *.NET*-процессе, имеющем доступ к файловой системе компьютера. *Blazor* позволяет создавать интерактивные интерфейсы с применением *C#* и *Razor*-компонентов, поддерживает серверный рендеринг и интерактивный режим, при котором события пользовательского интерфейса обрабатываются на стороне *.NET*-приложения *[7]*.

Такой вариант предпочтительнее отдельного настольного интерфейса для рассматриваемого проекта, поскольку обеспечивает доступ к интерфейсу через браузер и сохраняет единый стек *C#/.NET*. При этом ограничения браузерной песочницы не мешают наблюдению за каталогом: доступ к локальным файлам выполняет серверная часть клиентского приложения, а браузер используется только для отображения состояния и отправки пользовательских команд. Интерфейс включает настройку набора резервного копирования, создание общего набора и подключение к нему по коду, просмотр участников, файлов и версий, восстановление исторических состояний и просмотр журнала изменений.

Для хранения структурированных сведений выбрана реляционная СУБД *PostgreSQL* 17. В базе данных размещаются сведения о клиентах, наборах резервного копирования, участниках общих наборов, логических файлах, версиях и журнале изменений. Для общего набора дополнительно сохраняются режим доступа, код подключения, роль участника и сведения об авторе операции. Выбор *PostgreSQL* обусловлен поддержкой транзакций, ограничений целостности, индексов и многоверсионного управления конкурентным доступом. Официальная документация также указывает на наличие механизмов надёжности, восстановления и развитых средств конкурентной работы, что соответствует требованиям к серверной части системы *[8]*.

Доступ к *PostgreSQL* из *.NET* организуется через провайдер *Npgsql* и его интеграцию с *Entity Framework Core*. Провайдер *Npgsql.EntityFrameworkCore.PostgreSQL* предоставляет стандартную для *EF Core* модель *DbContext*, отображение сущностей на таблицы и поддержку миграций схемы базы данных. Это позволяет описывать структуру метаданных в виде типизированных классов *C#*, формировать ограничения и индексы сред-

ствами модели и изменять схему контролируемыми миграциями без написания большого объёма низкоуровневого *SQL*-кода *[9]*.

Бинарное содержимое резервных копий нецелесообразно хранить непосредственно в реляционной таблице. В разработанной системе *PostgreSQL* используется для метаданных, а версии файлов размещаются в отдельном файловом хранилище. Такое разделение упрощает потоковую запись и чтение больших файлов, сохраняет прозрачную структуру хранилища и не перегружает базу данных бинарными объектами.

В качестве основной среды разработки применяется *Microsoft Visual Studio*. Среда предоставляет редактор *C#*, сборку решений, отладчик, интеграцию с системой тестирования и средствами *Git*, что позволяет работать с многопроектным решением в едином интерфейсе. Документация *Visual Studio* отдельно выделяет возможности написания и редактирования кода, сборки, отладки, тестирования и открытия проектов из репозиториев, поэтому данная среда соответствует характеру разрабатываемого программного комплекса *[*10*]*.

Совместное использование *PostgreSQL*, файлового хранилища и локального веб-интерфейса позволяет разделить различные виды данных в соответствии с характером их обработки. Структурированные сведения доступны для выборки, фильтрации и контроля целостности средствами реляционной СУБД, тогда как содержимое файлов передаётся и сохраняется независимо в потоковом режиме. При этом пользователь получает единый интерфейс для управления резервным копированием, просмотра состояния общего набора, истории версий и результатов синхронизации. Выбранные средства не дублируют функции друг друга и образуют согласованный технологический стек, пригодный как для однопользовательского, так и для многоклиентского режима работы системы.

Разделение структурированных и бинарных данных положительно влияет и на надёжность программного комплекса. Метаданные позволяют определить, какие версии файлов должны существовать, их размеры, контрольные суммы и физическое расположение, тогда как файловое хранилище содержит непосредственно сохранённое содержимое. Благодаря этому состояние системы может проверяться без изменения пользовательских данных: сведения из базы данных сопоставляются с наличием соответствующих файловых объектов. Кроме того, независимость двух способов хранения упрощает сопровождение системы, поскольку изменение структуры метаданных не требует преобразования уже сохранённых бинарных версий, а операции с файлами не затрагивают структуру реляционных таблиц.

Итоговый набор выбранных технологий приведён в таблице 1.2.

Таблица 1.2 – Технологический стек программного комплекса

Назначение	**Выбранное средство**	**Причина выбора**
Язык и платформа	*C#*, *.NET* 9	Единая среда для клиентской и серверной частей
Сетевой транспорт	*TCP*, *System.Net.Sockets*	Надёжная передача данных и собственный протокол
Наблюдение за файлами	*FileSystemWatcher*	Событийное отслеживание изменений каталога
Управляющие сообщения	*System.Text.Json*	Сериализация типизированных *DTO* в *UTF*-8 *JSON*
Пользовательский интерфейс	*Blazor Web App*	Локальный веб-интерфейс на *C#* и *Razor*
Метаданные	*PostgreSQL* 17	Транзакции, целостность, индексы и конкурентный доступ
Доступ к данным	*EF Core* + *Npgsql*	Типизированная модель данных и миграции
Среда разработки	*Visual Studio*	Сборка, отладка и тестирование решения

Выбранный технологический стек соответствует требованиям задания и объединяет сетевые, файловые, пользовательские и серверные средства в единой среде *.NET*. *PostgreSQL* используется для согласованных метаданных, а отдельное файловое хранилище – для неизменяемых бинарных версий. Такое сочетание технологий создаёт основу для архитектуры и алгоритмов синхронизации, рассматриваемых далее.

Разделение метаданных и файлового содержимого также упрощает сопровождение системы. Операции поиска, фильтрации и получения истории выполняются по компактным структурированным записям *PostgreSQL*, тогда как чтение и запись резервных копий остаются последовательными файловыми операциями. В результате сервер метаданных не выполняет передачу крупных бинарных объектов, а сервис хранения может работать с потоками независимо от структуры реляционных таблиц. Такое распределение обязанностей соответствует выбранной клиент-серверной архитектуре и позволяет развивать каждый компонент без изменения пользовательского сценария.

Выбранные средства образуют согласованный стек без дублирующих технологий. *Blazor* используется только для взаимодействия с пользователем,

локальная часть клиента отвечает за доступ к файловой системе, *TCP* обеспечивает сетевой транспорт, а *PostgreSQL* и файловое хранилище выполняют различные задачи сохранения данных. Это разделение снижает связанность программных компонентов и создаёт основу для архитектуры, в которой сетевой шлюз координирует обращения к специализированным серверным сервисам. Архитектурное устройство системы и алгоритмы её работы рассматриваются в следующем разделе.

При выборе средств хранения учитывалась также возможность независимой проверки состояния данных. Реляционная модель позволяет однозначно определить зарегистрированные версии, их номера, размеры и контрольные суммы, а файловое хранилище – проверить наличие соответствующих бинарных объектов. Такое разделение упрощает диагностику: несогласованность между записью метаданных и физическим файлом может быть выявлена без вмешательства в содержимое уже зафиксированных версий. Для системы резервного копирования это важно, поскольку операции проверки должны быть неразрушающими и не должны изменять пользовательскую историю.

Использование локального веб-интерфейса также соответствует структуре приложения. Браузер отвечает только за отображение состояния и ввод команд пользователя, тогда как доступ к контролируемому каталогу, вычисление хеш-сумм и сетевой обмен выполняются локальной частью *.NET*-приложения. Благодаря этому не требуется предоставлять браузеру прямой доступ к произвольным каталогам файловой системы, а пользователь сохраняет привычный способ работы через веб-страницу.

Выбранный технологический стек также соответствует требованиям многоклиентского режима работы системы. Использование *TCP* позволяет организовать единый сетевой обмен между несколькими клиентскими рабочими станциями и серверными компонентами, а применение асинхронной модели *.NET* снижает влияние операций ввода-вывода на отзывчивость приложения. *PostgreSQL* обеспечивает централизованное хранение сведений о клиентах, общих наборах резервного копирования, участниках, версиях файлов и журнале изменений, тогда как файловое хранилище независимо обслуживает бинарное содержимое версий.

Таким образом, выбранные технологии обеспечивают не только реализацию основных функций резервного копирования, но и необходимую надёжность, сопровождаемость и возможность работы нескольких клиентов с общими данными. Архитектурное устройство системы и алгоритмы её работы рассматриваются в следующем разделе.

2 АРХИТЕКТУРА И АЛГОРИТМЫ СИСТЕМЫ РЕЗЕРВНОГО КОПИРОВАНИЯ

2.1 Функциональная структура и взаимодействие компонентов системы

Архитектура программного комплекса сформирована с учётом необходимости разделить пользовательский интерфейс, работу с локальной файловой системой, сетевое взаимодействие, обработку метаданных и физическое хранение резервных версий. Такое разделение позволяет каждому компоненту выполнять ограниченный набор обязанностей и уменьшает связанность между рабочей станцией пользователя и серверной частью. В системе выделяются локальные клиентские приложения, шлюз синхронизации, сервис метаданных и сервис файлового хранения.

Клиентское приложение запускается на каждой рабочей станции, участвующей в резервном копировании. Оно предоставляет локальный веб-интерфейс, имеет доступ к выбранному каталогу, отслеживает файловые события и поддерживает собственное локальное состояние синхронизации. Каждая рабочая станция регистрируется на сервере с постоянным идентификатором, поэтому действия разных клиентов могут быть однозначно различены даже при работе с одним и тем же набором файлов.

Система поддерживает два режима набора резервного копирования. Личный набор связан с одной рабочей станцией и используется как обычное централизованное резервное хранилище. Общий набор предназначен для нескольких клиентов. Его создатель становится владельцем и получает код подключения, по которому другие зарегистрированные рабочие станции могут присоединиться к тому же серверному состоянию. После присоединения каждый участник использует собственную локальную папку, но относительные пути, версии и журнал изменений относятся к одному общему набору.

Локальная синхронизация строится на сочетании наблюдения за файловой системой и периодического согласования с сервером. Наблюдатель обеспечивает быструю реакцию на действия пользователя в локальной папке, а серверная сверка позволяет получить изменения, выполненные на других рабочих станциях. Для предотвращения параллельного выполнения несовместимых операций синхронизация сериализуется: в один момент времени выполняется одна операция согласования или передачи изменения.

Все обращения клиентских приложений к серверной части проходят через шлюз синхронизации. Он является единой сетевой точкой входа, выполняет рукопожатие, поддерживает изолированные клиентские сессии и направляет

запросы в специализированные сервисы. Клиенты не обращаются напрямую ни к *PostgreSQL*, ни к каталогу физического хранилища, поэтому внутренняя организация серверной части остаётся скрытой от рабочих станций.

Сервис метаданных хранит логическое состояние системы: зарегистрированных клиентов, наборы резервного копирования, состав участников общего набора, сведения о файлах и их версиях, подготовленные операции и журнал изменений. Для каждой завершённой операции сохраняется информация о клиенте-инициаторе. Это позволяет определить не только время и тип изменения, но и рабочую станцию, с которой оно было выполнено.

Сервис хранения отвечает только за бинарное содержимое версий. Он принимает и выдаёт файловые потоки, проверяет размер и *SHA*-256, использует временную область при записи и публикует объект только после успешной проверки. Метаданные версии фиксируются после сохранения содержимого, поэтому серверное описание файла не должно ссылаться на незавершённую передачу.

Общее расположение компонентов и взаимодействие двух рабочих станций с серверной частью представлены на рисунке 2.1.

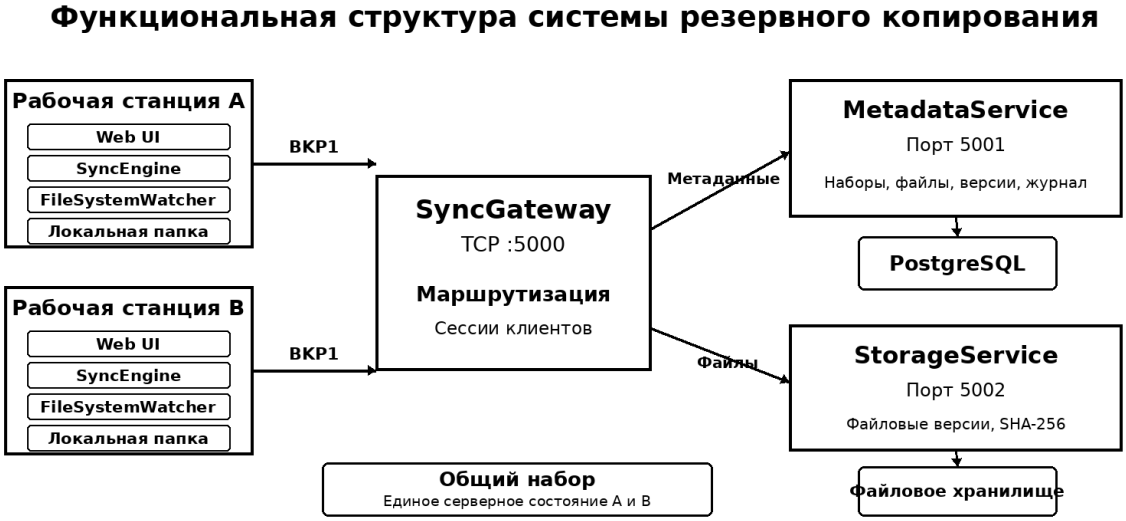


Рисунок 2.1 – Функциональная структура системы резервного копирования

В многоклиентском режиме единицей совместной работы является общий набор, а не локальная папка конкретного компьютера. Сервер проверяет принадлежность клиента к набору перед операциями чтения и изменения. Владелец и участники получают одинаковое актуальное состояние файлов, однако в журнале сохраняется фактический автор каждой операции. Такое построение позволяет централизованно хранить историю и одновременно поддерживать независимые локальные каталоги нескольких рабочих станций.

2.2 Организация сетевого обмена и модель общего хранения данных

Сетевое взаимодействие между компонентами построено поверх *TCP*. Согласно спецификации *RFC* 9293, *TCP* предоставляет надёжную упорядоченную передачу потока байтов, однако не задаёт границы прикладных сообщений. Поэтому в программном комплексе используется собственный *Protocol v*1, определяющий структуру кадра, правила идентификации сообщений и способ передачи управляющих данных и файлового содержимого.

Каждый кадр начинается с четырёх байтов, содержащих длину *JSON*-заголовка в формате *UInt*32 *Big-Endian*. Затем следует *UTF*-8 *JSON*-заголовок указанной длины, после чего передаётся тело размером *BodyLength* байт. Тело может содержать управляющий *JSON*-объект либо бинарное содержимое файла. Максимальный размер заголовка ограничен 64 КиБ, что предотвращает неконтролируемое выделение памяти при повреждённом или некорректном входном кадре.

JSON-заголовок содержит сигнатуру *Magic* со значением *BKP*1, номер версии протокола, тип сообщения, идентификатор *MessageId*, необязательный *CorrelationId*, тип содержимого, длину тела и время формирования *SentAtUtc*. *MessageId* идентифицирует запрос, а *CorrelationId* позволяет связать ответ с исходным сообщением. Одинаковая структура кадра применяется для рукопожатия, проверки соединения, операций с общими наборами, получения манифеста, передачи метаданных и потоковой загрузки файлов.

Структура сетевого кадра и основные поля *JSON*-заголовка показаны на рисунке 2.2.

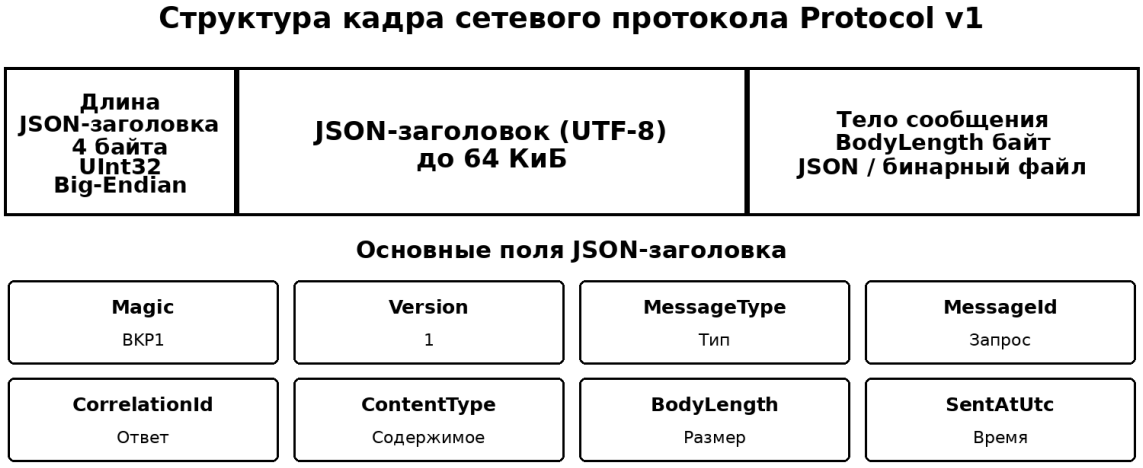


Рисунок 2.2 – Структура кадра сетевого протокола *Protocol v*1

*Protocol v*1 не предполагает мультиплексирование нескольких прикладных операций внутри одной клиентской *TCP*-сессии. После отправки запроса клиент полностью считывает заголовок и тело ответа и только затем начинает следующую операцию. Такой порядок особенно важен при чередовании коротких *JSON*-сообщений и бинарных файловых потоков: непрочитанный остаток предыдущего тела нарушил бы границы следующего кадра. Для больших файлов содержимое передаётся блоками, поэтому шлюзу не требуется размещать целый объект в оперативной памяти.

Логическая модель общего хранения строится вокруг клиента, набора резервного копирования, участника общего набора, файла, версии файла, подготовленной операции и записи журнала. Набор имеет владельца и режим доступа. Для общего набора дополнительно формируется код подключения и хранится состав участников с ролями «владелец» и «участник». После присоединения код больше не используется как средство авторизации каждой операции: доступ определяется зарегистрированным членством клиента в конкретном наборе.

Логический файл существует внутри набора независимо от конкретной рабочей станции. Для него сохраняются относительный путь, идентификатор, номер текущей версии и признак логического удаления. Переименование изменяет путь существующего файла и не должно уничтожать его историю. Удаление также является логической операцией: исторические версии сохраняются и остаются доступными для последующего восстановления.

Каждая успешно зафиксированная редакция представляет отдельную неизменяемую версию. Для неё хранятся порядковый номер, размер, *SHA*-256 и ссылка на бинарный объект в файловом хранилище. Временная подготовленная операция отделяет намерение создать новую версию от факта успешного сохранения содержимого. Благодаря уникальному идентификатору операции повторная фиксация после потери сетевого ответа не должна создавать дубликат версии.

Журнал изменений отражает завершённые действия над файлами и содержит сведения о наборе, файле, типе операции и фактическом клиенте-инициаторе. В общем наборе это принципиально важно: владелец серверного пространства и автор изменения могут быть разными рабочими станциями. Журнал используется для отображения последовательности создания, изменения, переименования, удаления, восстановления и сохранения конфликтных копий.

Для периодического согласования клиент получает серверный манифест общего набора. В нём отражается текущее состояние файлов: идентификатор,

относительный путь, номер версии, признак удаления и контрольная сумма. Сопоставление манифеста с локальным индексом позволяет определить изменения, выполненные другими участниками, и не создавать новых версий при отсутствии фактических различий.

Физическое хранилище отделено от реляционной модели. Каждой версии соответствует самостоятельный бинарный объект, а в *PostgreSQL* хранится только ссылка на его расположение и проверяемые характеристики. При записи сначала используется временный объект, затем проверяются размер и *SHA*-256, после чего файл атомарно переводится в опубликованное состояние. Такой порядок уменьшает риск появления метаданных версии без соответствующего содержимого.

2.3 Механизмы синхронизации, версионирования, разрешения конфликтов и восстановления файлов

Автоматическая синхронизация сочетает два источника информации. Локальные изменения обнаруживаются наблюдателем файловой системы, а изменения других рабочих станций – периодическим запросом серверного манифеста. При создании, изменении, переименовании или удалении файла локальное событие проходит короткую задержку и проверку фактического состояния, что позволяет объединить серию близких уведомлений и не создавать лишние версии.

Для идентификации фактического содержимого используется *SHA*-256. Стандарт *Secure Hash Standard* определяет *SHA*-256 как криптографический алгоритм формирования дайджеста сообщения. В системе хеш применяется как контрольный признак: он позволяет определить наличие реального изменения, проверить целостность принятого файла и убедиться, что восстановленная версия соответствует серверному объекту.

При работающем общем наборе локальные изменения передаются на сервер сразу после стабилизации файла, а изменения других участников обнаруживаются фоновым согласованием через заданный интервал. Повторная сверка не запускается параллельно с уже выполняющейся операцией синхронизации. Если серверный манифест совпадает с локальным индексом, цикл завершается без загрузок, новых версий и записей журнала.

Последовательность сохранения локального изменения и его распространения на другую рабочую станцию показана на рисунке 2.3.

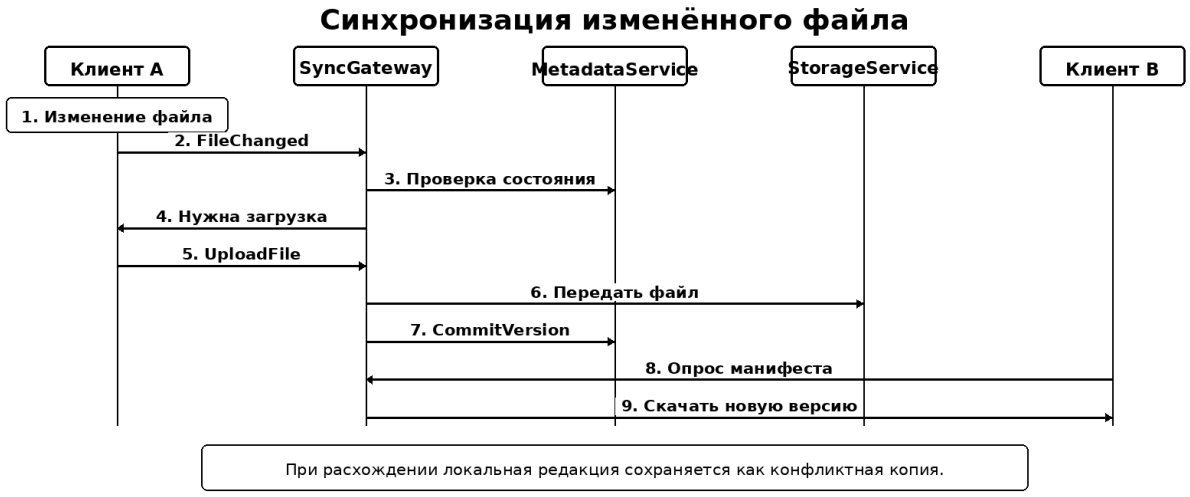


Рисунок 2.3 – Синхронизация изменённого файла в общем наборе

После вычисления размера и *SHA*-256 клиент передаёт сведения об изменении через шлюз. Сервис метаданных сопоставляет их с текущим состоянием файла. Если содержимое совпадает с уже сохранённой версией, передача бинарных данных не требуется. При фактическом изменении создаётся подготовленная операция и определяется номер следующей версии.

Содержимое файла передаётся бинарным потоком в сервис хранения. Шлюз не сохраняет полный объект у себя, а ретранслирует данные ограниченными блоками. Сервис хранения записывает временный файл, одновременно вычисляет фактические размер и *SHA*-256 и публикует объект только при совпадении ожидаемых характеристик. После этого сервис метаданных фиксирует новую версию, обновляет текущее состояние файла и добавляет запись в журнал. Повторная фиксация той же подготовленной операции является идемпотентной.

Другой участник общего набора получает новую версию при очередной сверке манифеста. Серверное содержимое загружается потоково и применяется к локальному каталогу с подавлением ожидаемого собственного события наблюдателя. Благодаря этому получение удалённого изменения не приводит к повторной отправке той же версии обратно на сервер.

Переименование и удаление не требуют копирования уже сохранённых бинарных версий. При зарегистрированном переименовании изменяется относительный путь того же логического файла, поэтому его идентификатор и история сохраняются. При удалении устанавливается логический признак удаления, а исторические версии остаются доступными. Эти изменения также распространяются на остальных участников общего набора через манифест.

Отдельная ситуация возникает, когда локальная рабочая станция изменила файл на основе одной серверной версии, а до передачи этого изменения другой участник уже зафиксировал более новую редакцию. Автоматическое построчное слияние содержимого в системе не выполняется. Вместо перезаписи одной из редакций локальное расходящееся содержимое сохраняется как отдельная конфликтная копия, а основному файлу применяется актуальная серверная версия. Конфликтная копия затем загружается в общий набор как самостоятельный файл и распространяется на остальных клиентов. Такой подход исключает незаметную потерю одной из пользовательских редакций.

При временной недоступности шлюза интерфейс клиентского приложения остаётся доступным, а локальные изменения сохраняются в файловой системе. Сетевые запросы завершаются ограниченной ошибкой, а повторное подключение выполняется фоново с увеличивающейся задержкой между попытками. После восстановления связи клиент повторно выполняет рукопожатие и запускает согласование, в ходе которого передаются накопленные локальные изменения и загружаются изменения других участников.

Восстановление исторической версии выполняется так, чтобы сохранить существующую историю и не уничтожить несинхронизированные локальные изменения. Общая последовательность восстановления приведена на рисунке 2.4.

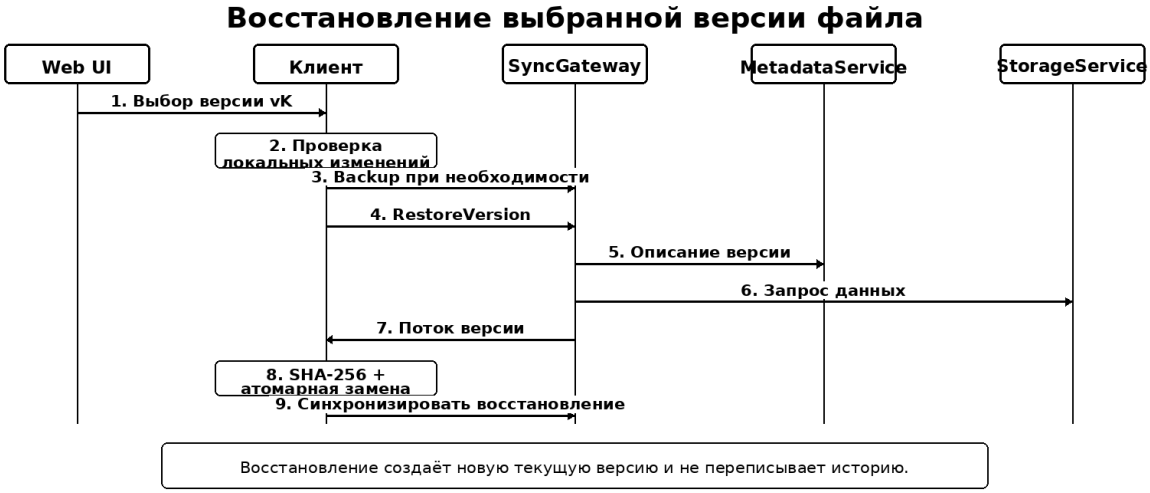


Рисунок 2.4 – Восстановление выбранной версии файла

Перед заменой локального файла клиент проверяет его текущее состояние. Если обнаружены несинхронизированные изменения, они предварительно передаются на сервер как очередная версия. Затем шлюз получает описание выбранной исторической редакции, запрашивает соответствующий бинарный объект у сервиса хранения и потоково передаёт его клиенту.

Полученные данные записываются во временный файл в том же каталоге. После завершения загрузки проверяются размер и *SHA*-256. При несовпадении временный объект удаляется, а исходный файл остаётся неизменным. После успешной проверки регистрируется подавление ожидаемого события наблюдателя и выполняется атомарная замена целевого файла.

Восстановление не уменьшает номер текущей версии и не переписывает предыдущую историю. Восстановленное содержимое синхронизируется с сервером как новое актуальное состояние, получает следующий номер версии и отдельную запись действия восстановления. Если файл ранее был логически удалён, восстановление создаёт его локально заново и снимает признак удаления. В общем наборе новое состояние затем распространяется на другие рабочие станции обычным механизмом согласования.

Дополнительная надёжность восстановления обеспечивается тем, что операция не изменяет ранее сохранённые версии и выполняется только после проверки полученного содержимого. Клиент использует метаданные выбранной версии для контроля размера и контрольной суммы, поэтому повреждённые или неполностью переданные данные не заменяют существующий локальный файл. Такой подход особенно важен при работе нескольких клиентов с общим набором: восстановление, выполненное одним участником, фиксируется как новое серверное состояние и далее распространяется на остальные рабочие станции тем же механизмом синхронизации. В результате история изменений остаётся последовательной, а каждое восстановление может быть однозначно прослежено по номеру версии и записи журнала.

Таким образом, алгоритмы синхронизации объединяют событийное отслеживание локальных изменений, периодический серверный манифест, неизменяемое версионирование, безопасное разрешение конфликтов и автоматическое восстановление после сетевых сбоев. Сервер остаётся источником общего состояния, а локальный индекс позволяет каждому клиенту определить, какие изменения требуется передать или получить.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ КЛИЕНТ-СЕРВЕРНОЙ СИСТЕМЫ

3.1 Структура решения и общие программные компоненты

Программный комплекс реализован как решение *.NET* 9, состоящее из шести проектов. Такое разбиение напрямую соответствует архитектуре, рассмотренной во второй главе: клиентское приложение, шлюз синхронизации, сервис метаданных и сервис файлового хранения развиваются как отдельные исполняемые компоненты, общие сетевые контракты вынесены в библиотеку, а автоматизированные проверки собраны в самостоятельном тестовом проекте.

Проект *Backup.Client.Web* содержит локальное *Blazor*-приложение, которое одновременно предоставляет веб-интерфейс и выполняет файловые операции на рабочей станции. В нём размещены *Razor*-компоненты страниц, сервис постоянной идентификации клиента, хранение профиля синхронизации, локальный индекс файлов, наблюдение за каталогом, сканирование файловой системы, механизм подавления служебных событий и основной движок синхронизации.

Проект *Backup.SyncGateway* реализует единую *TCP*-точку входа на порту 5000. Он принимает клиентские соединения, выполняет *Hello*-рукопожатие, создаёт независимую сессию для каждого клиента и маршрутизирует управляющие запросы в *MetadataService* либо потоковые операции в *StorageService*. Клиентские рабочие станции не обращаются к внутренним сервисам и *PostgreSQL* напрямую.

Backup.MetadataService содержит прикладную логику метаданных, сущности *Entity Framework Core*, *MetadataDbContext* и миграции *PostgreSQL*. *Backup.StorageService* отвечает только за бинарные версии файлов: принимает поток, сохраняет временный объект, проверяет размер и *SHA*-256, публикует завершённую версию и выдаёт её при восстановлении.

Библиотека *Backup.Shared* используется всеми сетевыми компонентами. В ней определены типы сообщений *Protocol v*1, *DTO* запросов и ответов, структура кадра, потоковые вспомогательные классы, валидация относительных путей и общие типы режима набора. После добавления многоклиентского режима здесь также размещены контракты подключения к общему набору, получения списка участников и перечисление *BackupSetAccessMode* со значениями *Private* и *Shared*.

Проект *Backup.Tests* содержит модульные, интеграционные, сквозные и регрессионные проверки. Тестовый проект использует те же общие контракты, что и рабочие компоненты, поэтому ошибки кадрирования, повторной фикса-

ции версии, восстановления, общего доступа, автоматической синхронизации и восстановления соединения проверяются на уровне реального протокольного взаимодействия.

Структура решения и регистрация основных сервисов клиентского приложения в *Visual Studio* показаны на рисунке 3.1.

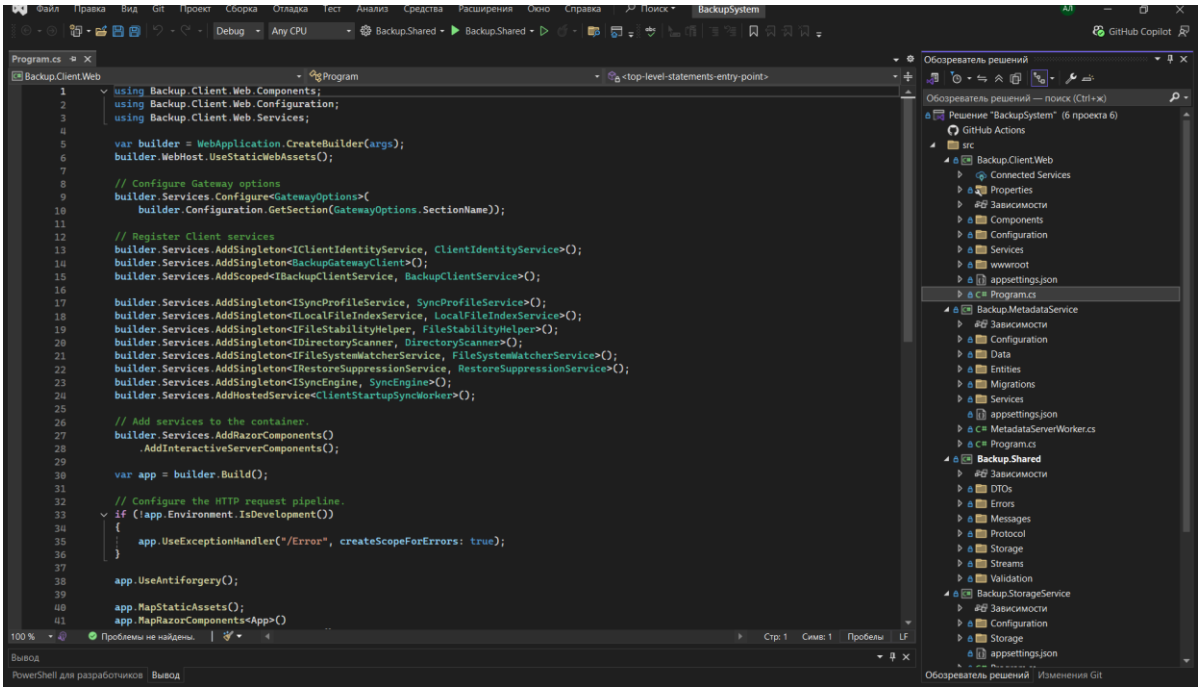


Рисунок 3.1 – Структура программного решения в *Visual Studio*

В точке входа *Backup.Client.Web* используется встроенный контейнер зависимостей *.NET*. Основные сервисы синхронизации регистрируются как *singleton*, чтобы одна и та же рабочая станция использовала единый *ClientId*, один активный профиль, общий локальный индекс и один экземпляр *SyncEngine* на протяжении всего времени работы приложения. Страницы интерфейса получают доступ к этим сервисам через внедрение зависимостей.

Конфигурация вынесена в *appsettings.json*, классы *Options* и аргументы командной строки. Отдельный параметр *ClientDataRoot* определяет каталог локального состояния клиента. Благодаря этому в демонстрационном режиме два процесса *Backup.Client.Web* могут работать одновременно с независимыми файлами *client-state.json*, *sync-profile.json* и локальными индексами, сохраняя разные *ClientId* и имена рабочих станций.

Серверные проекты запускаются на базе *Generic Host*. Адреса *MetadataService*, *StorageService* и *SyncGateway* задаются конфигурацией, а общие *DTO* остаются единственным контрактом между процессами. Такое построение поз-

воляет изменять внутреннюю реализацию отдельного сервиса без изменения формата взаимодействия остальных компонентов.

3.2 Реализация клиентского приложения и шлюза синхронизации

Клиентская часть реализована как *Blazor Web App* в интерактивном серверном режиме. Браузер отвечает за отображение состояния и передачу пользовательских команд, а доступ к локальным каталогам, вычисление контрольных сумм, наблюдение за файлами и *TCP*-взаимодействие выполняются локальным *.NET*-процессом. Поэтому веб-интерфейс не ограничивает работу системы с произвольной папкой на рабочей станции.

Постоянная идентификация реализована сервисом *ClientIdentityService*. При первом запуске создаётся *ClientId* и вместе с именем рабочей станции сохраняется в *client-state.json*. Запись выполняется атомарно через временный файл. Для второго демонстрационного клиента используется отдельный *ClientDataRoot*, поэтому *Client A* и *Client B* имеют независимые идентификаторы и могут одновременно подключаться к одному общему набору.

Профиль синхронизации хранит идентификатор набора, локальный путь, режим доступа, код подключения и признак автоматической синхронизации. Для общего набора владелец создаёт *Shared*-профиль и получает *ShareCode* формата *XXXX-XXXX*. Другой клиент вводит этот код и собственный локальный каталог, после чего выполняется *JoinSharedBackupSet* и создаётся локальный профиль участника. Режим *Shared* сохраняется как в памяти, так и в *JSON*-профиле, чтобы после перезапуска не происходило ошибочного возврата в *Private*.

Состояние общего набора на рабочей станции участника показано на рисунке 3.2.

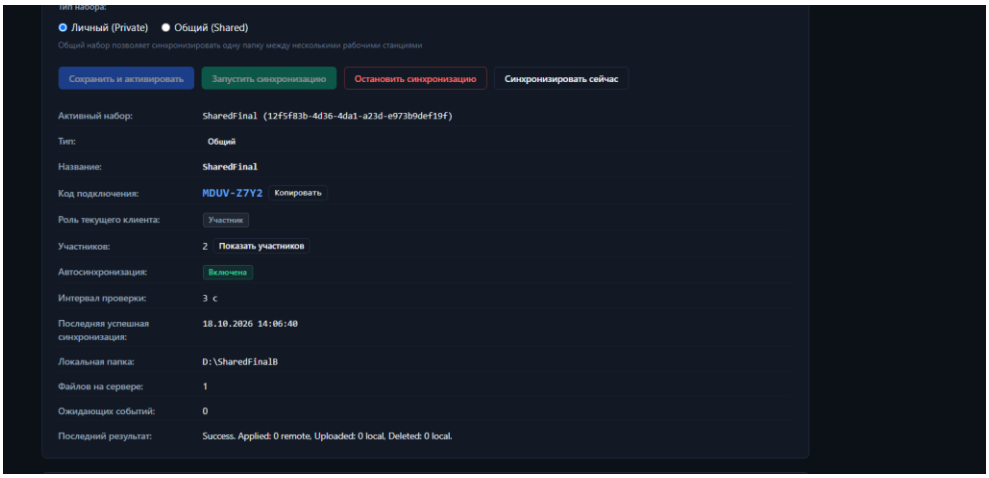


Рисунок 3.2 – Сведения об общем наборе на рабочей станции участника

На странице общего набора отображаются код подключения, роль текущего клиента, число участников, состояние автосинхронизации, интервал фоновой проверки, время последней успешной сверки и локальный каталог. Кнопка копирования позволяет передать *ShareCode* другому пользователю без ручного набора.

Автоматическая синхронизация сосредоточена в *SyncEngine*. После запуска профиль проходит первичное согласование, затем активируется *FileSystemWatcherService*. События *Created*, *Changed*, *Renamed* и *Deleted* помещаются во внутреннюю очередь, объединяются короткой задержкой и обрабатываются только после стабилизации файла. Для нового либо изменённого содержимого вычисляются размер и *SHA*-256, после чего сведения передаются серверу.

Для общего набора дополнительно запускается один фоновый цикл периодического согласования. Интервал задаётся параметром *SharedSyncIntervalSeconds*; в обычной конфигурации используется несколько секунд, а демонстрационный скрипт задаёт три секунды. Цикл запрашивает манифест сервера и сравнивает его с *LocalFileIndexService*. Если различий нет, никакие версии и записи журнала не создаются.

Параллельное выполнение нескольких сверок предотвращается семафором синхронизации. Если в момент фонового опроса выполняется отправка локального изменения либо ручная команда синхронизации, очередная итерация пропускается. Это исключает наложение операций над одним и тем же профилем и предотвращает образование лишних версий.

При применении удалённых изменений используется *RestoreSuppressionService*. Перед записью или удалением файла клиент регистрирует ожидаемое изменение. Если *FileSystemWatcher* получает соответствующее событие, оно распознаётся как служебное и не отправляется обратно на сервер. Для изменённого содержимого сравнивается *SHA*-256, поэтому реальное пользовательское редактирование не подавляется.

Если локальный файл был изменён на основе устаревшей серверной версии, автоматическое слияние текста не выполняется. *SyncEngine* сохраняет локальную редакцию под именем конфликтной копии, применяет актуальную серверную версию к основному файлу и затем загружает конфликтную копию как самостоятельный файл. Таким образом обе пользовательские редакции остаются доступными.

Страница сведений о файле отображает идентификатор, локальный путь, размер, текущий *SHA*-256 и последовательность сохранённых версий. Пользователь может выбрать историческую версию и запустить восстановление. Пример страницы с двумя сохранёнными версиями приведён на рисунке 3.3.

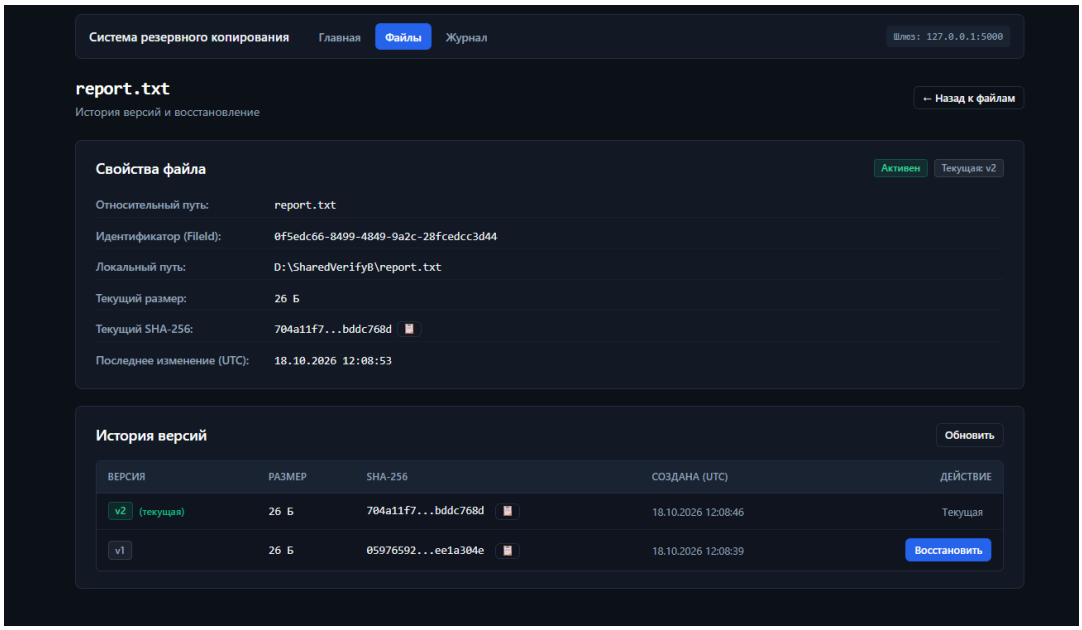


Рисунок 3.3 – История версий файла и операция восстановления

Взаимодействие с сервером инкапсулировано в *BackupGatewayClient*. Перед прикладными командами клиент устанавливает *TCP*-соединение, выполняет *Hello*-рукопожатие и использует *ProtocolFrameReader* и *ProtocolFrameWriter*. Поскольку одна *TCP*-сессия не мультиплексирует независимые прикладные операции, отправка запроса и полное чтение ответа сериализуются с помощью *SemaphoreSlim*.

Механизм переподключения сделан неблокирующим для интерфейса. Одна попытка *TCP*-подключения ограничена таймаутом, задержки повторных попыток выполняются вне блокировки соединения, а одновременно допускается только один фоновый *reconnect-loop*. При недоступном шлюзе страницы быстро возвращают понятное состояние ошибки и сохраняют возможность навигации.

После восстановления связи *BackupGatewayClient* выполняет новое рукопожатие и формирует событие *Reconnected*. *SyncEngine* помечает профиль как требующий согласования и выполняет внеочередную сверку. Накопившиеся локальные изменения передаются на сервер, а изменения других участников загружаются на рабочую станцию без ручного нажатия кнопки «Синхронизировать сейчас».

Перед восстановлением клиент проверяет локальное состояние файла. Несинхронизированное содержимое предварительно резервируется, выбранная серверная версия скачивается во временный файл, после чего проверяются размер и *SHA*-256. Только после успешной проверки выполняется атомарная заме-

на. Восстановленное содержимое затем синхронизируется как новая текущая версия, поэтому существующая история не переписывается.

На стороне *SyncGateway* каждое клиентское соединение обслуживается отдельной сессией. *GatewaySessionHandler* разбирает *MessageType* и передаёт запрос соответствующему обработчику. Помимо базовых операций резервного копирования реализованы сообщения присоединения к общему набору и получения его участников. Шлюз не содержит *EF Core*-контекста и не обращается к физическому каталогу хранения напрямую.

При загрузке файла шлюз хранит только контекст операции и ретранслирует бинарное тело ограниченными блоками в *StorageService*. Аналогично при скачивании версия передаётся потоком в обратном направлении. Благодаря этому размер передаваемого файла не определяет объём оперативной памяти, необходимый шлюзу.

Журнал изменений запрашивается клиентом через тот же шлюз. Для каждой записи отображаются тип действия, файл, состояние и фактическая рабочая станция-инициатор. В общем наборе это позволяет различать операции владельца и участника даже при работе с одним логическим файлом. Пример журнала, содержащего действия *Client A* и *Client B*, приведён на рисунке 3.4.

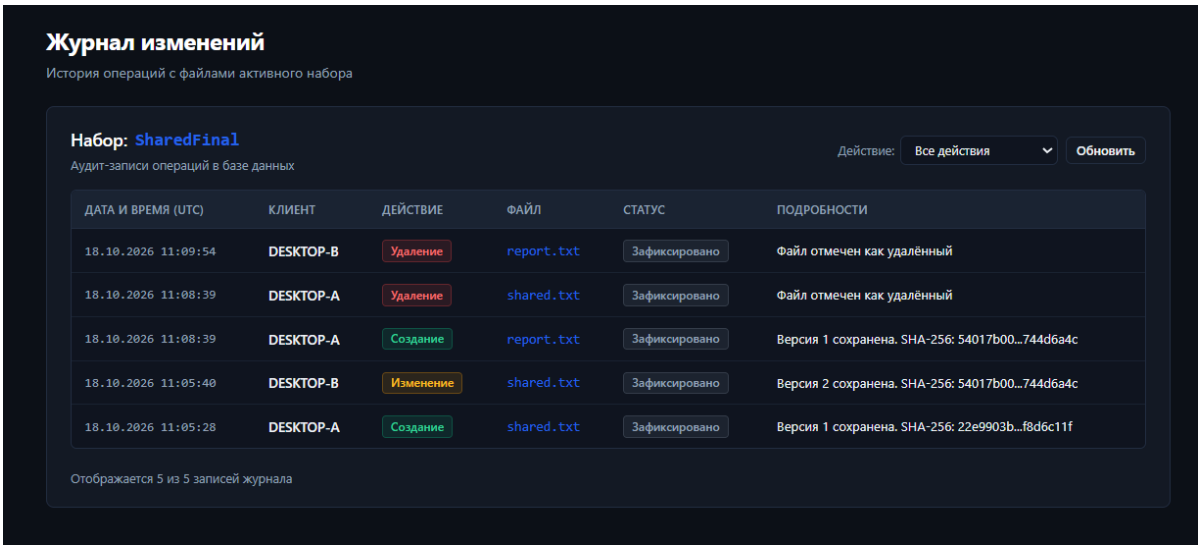


Рисунок 3.4 – Журнал изменений общего набора с указанием клиента-автора

Для конфликтной копии в журнале используется отдельный тип действия, поэтому сохранение расходящейся редакции отличается от обычного создания файла. Это позволяет при разборе истории определить момент возникновения конфликта, рабочую станцию, на которой была сохранена локальная редакция, и имя созданного конфликтного файла.

3.3 Реализация сервисов метаданных и файлового хранилища

Сервис метаданных построен вокруг *MetadataDbContext* и *MetadataApplicationService*. В итоговой модели используются сущности *ClientEntity*, *BackupSetEntity*, *BackupSetMemberEntity*, *FileEntity*, *FileVersionEntity*, *PendingFileOperationEntity* и *ChangeLogEntity*. Для внешних ключей, относительных путей, кодов подключения и идентификаторов операций настроены ограничения и индексы, а изменение схемы базы данных выполняется миграциями *EF Core*.

BackupSetEntity хранит владельца набора, имя, режим доступа и необязательный *ShareCode*. Для *Private*-набора код отсутствует. При создании *Shared*-набора *ShareCodeHelper* формирует код вида *XXXX-XXXX* из набора однозначно различимых символов. В таблице участников создатель получает роль *OWNER*, а подключившиеся клиенты — *MEMBER*. Повторное присоединение того же клиента является идемпотентным.

Перед чтением или изменением общего набора *MetadataApplicationService* проверяет принадлежность *ClientId* к списку участников. Сторонний зарегистрированный клиент не может обращаться к файлам набора и получает ошибку *ACCESS_DENIED*. Такая проверка выполняется на сервере и не зависит от состояния пользовательского интерфейса.

В журнале изменений поле владельца набора отделено от *ActorClientId*. Поэтому физическое пространство хранения остаётся связано с владельцем, но аудит показывает фактическую рабочую станцию, создавшую, изменившую, переименовавшую, удалившую или восстановившую файл. *ActorClientId* также переносится через подготовленную операцию от этапа подготовки версии к этапу фиксации.

Ключевая операция версионирования начинается с подготовки новой версии. Сервис находит логический файл по набору и относительному пути, сравнивает переданные размер и *SHA*-256 с текущим состоянием и определяет необходимость загрузки. Для нового содержимого создаётся *PendingFileOperation* с *OperationId* и ожидаемым номером версии. Подготовленная операция отделяет намерение изменить файл от факта успешного сохранения бинарных данных.

После сохранения содержимого выполняется *CommitVersion*. В одной транзакции создаётся *FileVersionEntity*, обновляется *CurrentVersionNumber* и формируется запись журнала. *OperationId* имеет уникальное ограничение, поэтому повторный *CommitVersion* после потери сетевого ответа возвращает уже существующий результат и не создаёт дополнительную версию.

Для участника общего набора физическое расположение версии всегда формируется в пространстве владельца. *StorageKey* имеет вид *clients*/{*OwnerClientId*}/*backup-sets*/{*BackupSetId*}/*files*/{*FileId*}/*versions*/{*VersionNumber*}.*bin*. Таким образом все участники работают с одним серверным набором, а не создают отдельные копии одного и того же хранилища по собственным *ClientId*.

Переименование изменяет *RelativePath* существующего *FileEntity* и сохраняет *FileId* и историю версий. Удаление реализовано логически через *IsDeleted*, поэтому уже опубликованные *FileVersionEntity* и бинарные объекты остаются доступными. Восстановление создаёт следующую версию, снимает признак удаления при необходимости и добавляет действие *RESTORE*.

StorageService не использует *PostgreSQL*. При загрузке он создаёт временный *.part*-файл, последовательно принимает поток и одновременно вычисляет *SHA*-256. После завершения проверяются фактический размер и контрольная сумма. Несовпадающий временный объект удаляется, а успешный файл атомарно перемещается в постоянный каталог версии.

При скачивании *StorageService* открывает только запрошенную неизменяемую версию и передаёт её потоком через *SyncGateway*. Служебные очистители работают исключительно с просроченными подготовленными операциями и временными файлами, не затрагивая опубликованные версии. Дополнительный проверяющий компонент может сопоставить записи *FileVersionEntity* с существующими *StorageKey* без изменения данных.

Для обычного запуска используется *start.ps*1, который последовательно запускает *MetadataService*, *StorageService*, *SyncGateway* и один экземпляр *Client.Web*. Для демонстрации общего хранилища предусмотрен *start-shared-demo.ps*1: он поднимает те же серверные компоненты и два клиента с разными *HTTP*-портами, *ClientDataRoot* и именами рабочих станций. Это позволяет на одном компьютере воспроизвести взаимодействие двух независимых пользователей.

Таким образом, программная реализация сохраняет разделение обязанностей, заложенное на архитектурном уровне. Клиент отвечает за пользовательский интерфейс и локальное состояние, *SyncGateway* – за *TCP*-сессии и координацию, *MetadataService* – за согласованное логическое состояние и права участников, а *StorageService* – за неизменяемые бинарные версии. Такая структура обеспечивает как обычное резервное копирование одной рабочей станции, так и автоматическую синхронизацию общего набора между несколькими клиентами.

4 ТЕСТИРОВАНИЕ И ВЕРИФИКАЦИЯ ПРОГРАММНОГО КОМПЛЕКСА

4.1 Методика и условия тестирования программного комплекса

Тестирование выполнялось после завершения реализации общего файлового хранилища, автоматической двусторонней синхронизации и механизмов восстановления после сетевых сбоев. Проверка проводилась на нескольких уровнях: сборка всего решения, модульные и интеграционные тесты, сквозные сценарии с реальными *TCP*-соединениями, регрессионные проверки пользовательского интерфейса и ручная проверка взаимодействия двух клиентских рабочих станций.

В качестве финальной программной версии использован коммит *e*74*d*6*b*1 с сообщением «*fix*: *stabilize live file details regression test*». Перед испытаниями рабочее дерево *Git* находилось в чистом состоянии: незакоммиченные изменения отсутствовали. Ветка *main* опережала удалённую *origin*/*main* на локальные коммиты, что не влияло на воспроизводимость проверки. Финальное состояние репозитория показано на рисунке 4.1.

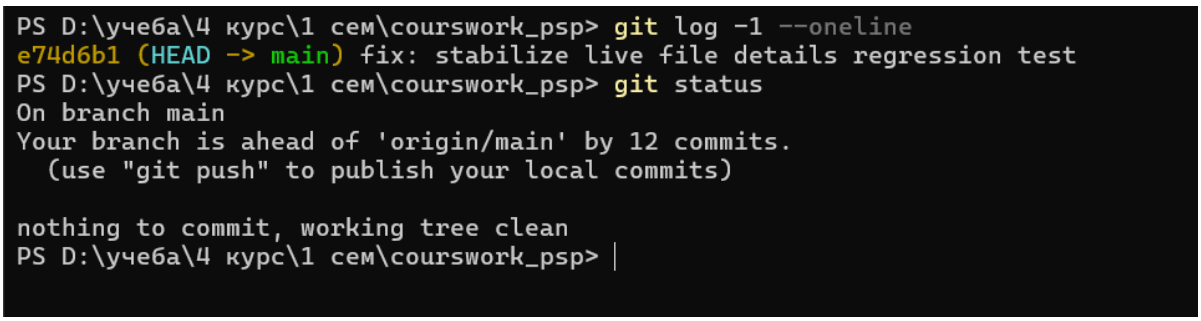


Рисунок 4.1 – Состояние репозитория перед финальной проверкой

Основные условия испытаний сведены в таблицу 4.1. Все серверные компоненты и оба демонстрационных клиента запускались локально, что позволяло контролировать сетевые отказы и повторные подключения без влияния внешней инфраструктуры.

Таблица 4.1 – Условия тестирования программного комплекса

Параметр	**Используемое значение**
Операционная среда	*Windows* 11, *x*64
Платформа	*C# / .NET* 9, *SDK* 9.0.318
СУБД	*PostgreSQL* 17
Серверные порты	*SyncGateway* – 5000; *MetadataService* – 5001; *StorageService* – 5002
Клиентские экземпляры	*Client A* – 5100; *Client B* – 5200
Тестовая инфраструктура	*xUnit*, *dotnet test*, интеграционные и *E*2*E*-сценарии
Каталоги общей синхронизации	*D*:*SharedVerifyA* и *D*:*SharedVerifyB*

Автоматические тесты разделены по назначению. Модульные проверки контролируют отдельные преобразования, валидацию кодов подключения, обработку путей, вычисление контрольных сумм и логику внутренних сервисов. Интеграционные тесты запускают взаимодействующие компоненты и проверяют работу протокола, *PostgreSQL*, физического хранилища и маршрутизацию через *SyncGateway*. Сквозные тесты воспроизводят последовательность действий реального клиента от регистрации до загрузки, восстановления и повторного согласования.

Регрессионные тесты добавлялись для дефектов, обнаруженных в ходе разработки: неправильного чтения остатка *TCP*-кадра, переключения *Shared*-профиля в *Private*, запуска двух клиентов с разными *ClientDataRoot*, зависания интерфейса при недоступном шлюзе и некорректной привязки теста страницы версий к активному профилю. Такой подход позволил закрепить исправления и исключить повторное появление уже найденных ошибок.

Ручная функциональная проверка выполнялась на общем наборе с двумя независимыми клиентами. Для каждого клиента использовались собственные *ClientId* и локальные каталоги, а изменения передавались через единственный серверный набор. При проверке отказоустойчивости процесс *SyncGateway* принудительно останавливался и запускался повторно, после чего контролировались отзывчивость интерфейса, автоматическое переподключение и догоняющая синхронизация.

4.2 Результаты автоматизированного и интеграционного тестирования

Первым этапом финальной верификации выполнялась сборка всего решения командой *dotnet build BackupSystem.sln*. Успешная сборка всех шести про-

ектов подтверждает согласованность ссылок между проектами, корректность текущей схемы исходного кода и отсутствие ошибок компиляции. Результат финальной сборки приведён на рисунке 4.2.

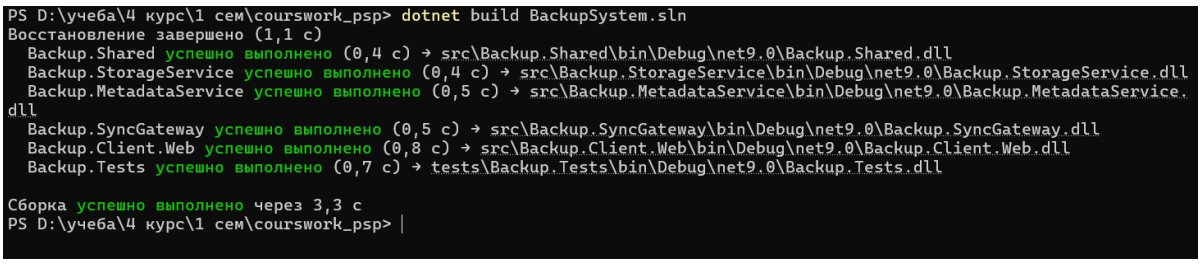


Рисунок 4.2 – Результат финальной сборки решения

Сборка завершилась успешно за 3,3 с. Предупреждения и ошибки компилятора отсутствовали. После этого без повторной сборки выполнялся полный набор автоматизированных тестов. Основные направления автоматической проверки приведены в таблице 4.2.

Таблица 4.2 – Основные направления автоматизированного тестирования

Группа	**Проверяемые свойства**	**Результат**
*Protocol v*1	Кадрирование *TCP*, сериализация, полное чтение тела ответа	Пройдено
Метаданные	Версионирование, идемпотентный *CommitVersion*, журнал	Пройдено
Хранилище	*Streaming*, *SHA*-256, временный *.part* и публикация версии	Пройдено
Shared Storage	*OWNER*/*MEMBER*, *ShareCode*, *ACCESS_DENIED*, общий *namespace*	Пройдено
Синхронизация	*CREATE*, *MODIFY*, *RENAME*, *DELETE*, *restore*, *conflict-copy*	Пройдено
Отказоустойчивость	*Offline UI*, *reconnect*, *catch-up*, отсутствие *duplicate loops*	Пройдено
Регрессии *UI*	Два *ClientDataRoot*, *FileDetails*, статические ресурсы *Blazor*	Пройдено

Полный запуск выполнялся командой *dotnet test BackupSystem.sln --no-build*. Итоговый набор содержит 339 тестов. Все 339 проверок завершились успешно, пропущенные и упавшие тесты отсутствовали; длительность финального прогона составила 27,2 с. Результат показан на рисунке 4.3.

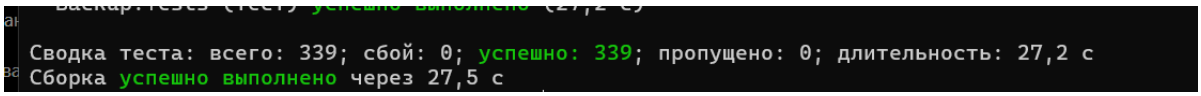


Рисунок 4.3 – Итог полного автоматизированного тестирования

Особое внимание уделялось стабильности регрессионных проверок. Тест *Live_PopulateSmokeFilesAndVerifyUiDetailsPage* первоначально использовал каталог *data*/*client*, тогда как запущенный демонстрационный *Client A* работал с *data*/*client-a*. Из-за этого тест создавал версии в одном наборе, а *Web UI* искал файл в другом активном профиле. После устранения зависимости от захардкоженного *ClientDataRoot* проверка стала использовать фактический активный профиль и дополнительно проверять наличие версий через серверный *API*.

Исправленный тест был выполнен двадцать раз подряд и во всех двадцати запусках завершился успешно. После этого два последовательных полных прогона также дали 339 успешных тестов из 339. Следовательно, финальный результат не является случайным единичным прохождением, а подтверждает воспроизводимое состояние тестового набора.

Интеграционные сценарии охватывают запуск *MetadataService*, *StorageService* и *SyncGateway*, регистрацию нескольких клиентов, создание *Shared*-набора, присоединение участника, передачу файлов, проверку физического *StorageKey*, авторства операций и отказ в доступе постороннему клиенту. Отдельно проверяется передача крупных файлов потоком, работа нескольких клиентов и автоматическое восстановление синхронизации после возврата шлюза.

4.3 Проверка функциональности и отказоустойчивости системы

Ручная проверка проводилась на общем наборе *SharedVerify*. *Client A* выступал владельцем, *Client B* подключался к тому же набору как участник. На обеих рабочих станциях была включена автоматическая синхронизация с интервалом фоновой сверки несколько секунд. Основные сценарии и фактически полученные результаты приведены в таблице 4.3.

Таблица 4.3 – Результаты ручной функциональной проверки

Сценарий	**Фактический результат**
1	2
Создание файла *Client A*	Файл автоматически появился у *Client B*; создана версия *v*1
Изменение файла *Client B*	Новая версия автоматически получена *Client A*; создана *v*2

Продолжение таблицы 4.3

1	2
Переименование	Новое имя распространилось на второй клиент, *FileId* и история сохранены
Удаление	Файл логически удалён на сервере и автоматически удалён у второго клиента
Восстановление *v*1	Содержимое *v*1 восстановлено и зафиксировано как новая текущая версия
Конкурентное изменение	Локальная редакция сохранена как *conflict-copy*, данные не потеряны
Отключение *SyncGateway*	*Web UI* остался отзывчивым и показал состояние «шлюз недоступен»
Возврат *SyncGateway*	Произошли автоматический *reconnect* и *catch-up* без ручной синхронизации

Журнал общего набора позволяет сопоставить действия с конкретной рабочей станцией. В нём фиксируются создание, изменение, переименование, удаление, восстановление и создание конфликтной копии. Для изменения, выполненного *Client B*, в поле клиента отображается *DESKTOP-B*, а операции *Client A* связываются с его рабочей станцией. Пример последовательности операций приведён на рисунке 4.4.

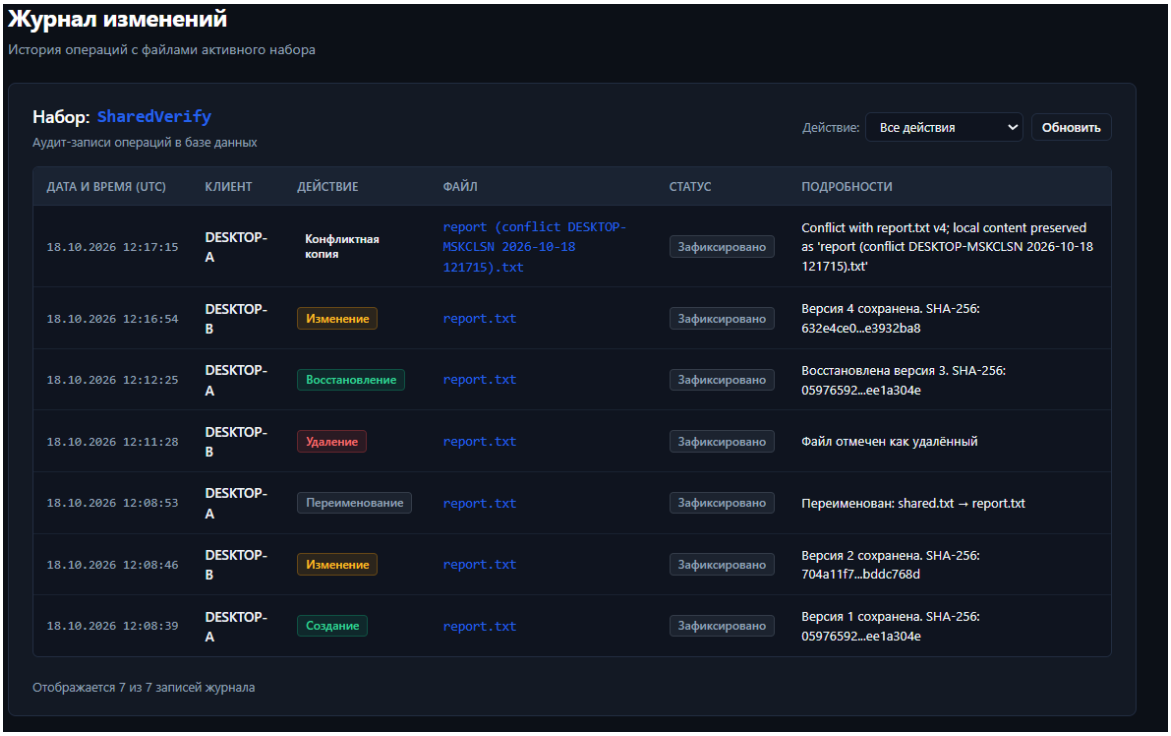


Рисунок 4.4 – Журнал ручной проверки операций общего набора

Отдельно проверялась обработка расходящихся изменений. При наличии локальной редакции, основанной на устаревшей серверной версии, система не выполняет автоматическое слияние и не перезаписывает пользовательские данные. Локальное содержимое сохраняется под новым именем, включающим признак *conflict*, имя рабочей станции и время возникновения ситуации. Основной файл после этого приводится к актуальному серверному состоянию.

Наличие одновременно основного файла и конфликтной копии в локальном каталоге показано на рисунке 4.5. После последующей фоновой синхронизации конфликтная копия появляется и у второго клиента как самостоятельный файл.

```
PS D:\учеба\4 курс\1 сем\courswork_psp> Get-Content "D:\SharedVerifyA\report.txt"
Конфликтная версия от клиента A
PS D:\учеба\4 курс\1 сем\courswork_psp> Set-Content "D:\SharedVerifyB\report.txt" "Основная версия от клиента B" -Encoding UTF8
PS D:\учеба\4 курс\1 сем\courswork_psp> Start-Sleep -Seconds 7
PS D:\учеба\4 курс\1 сем\courswork_psp> Get-Content "D:\SharedVerifyB\report.txt"
Основная версия от клиента B
PS D:\учеба\4 курс\1 сем\courswork_psp> Start-Sleep -Seconds 10
PS D:\учеба\4 курс\1 сем\courswork_psp> Get-ChildItem "D:\SharedVerifyA"


    Каталог: D:\SharedVerifyA


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10/18/2026   3:16 PM             62 report (conflict DESKTOP-MSKCLSN 2026-10-18 121715).txt
-a----        10/18/2026   3:17 PM             56 report.txt
```

Рисунок 4.5 – Сохранение конфликтной копии локального файла

Для проверки отказоустойчивости процесс *SyncGateway* был принудительно остановлен при работающих клиентах. В таком состоянии переход между страницами оставался доступным, а запросы к серверным данным завершались ограниченно по времени с понятным сообщением о недоступности шлюза. Фоновые циклы не создавали непрерывного потока повторных запросов и не блокировали *Blazor-circuit*.

Во время недоступности шлюза на *Client B* был создан файл *offline.txt*. До восстановления соединения он оставался только в локальном каталоге. После повторного запуска *SyncGateway* клиенты автоматически выполнили *Hello*-рукопожатие, восстановили соединение и запустили внеочередное согласование. Файл был загружен на сервер и без ручной команды появился в каталоге *Client A*.

Состояния интерфейса при недоступном и восстановленном шлюзе сопоставлены на рисунке 4.6. После восстановления система отображает подключённый шлюз, работающую синхронизацию и состояние «Синхронизировано».

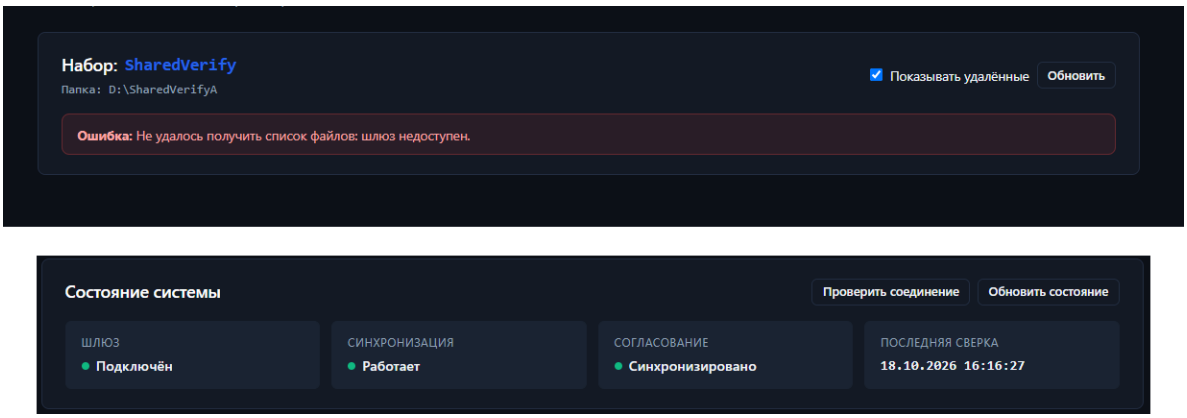


Рисунок 4.6 – Поведение клиента при отключении и восстановлении *SyncGateway*

Ручные испытания подтвердили соответствие поведения системы ожидаемым сценариям. Изменения распространяются в обоих направлениях, история версий сохраняется при переименовании и восстановлении, удаление не уничтожает архивные версии, а конфликтная ситуация не приводит к потере одной из редакций. При сетевом отказе пользовательский интерфейс остаётся работоспособным, а после возврата шлюза согласование выполняется автоматически.

Совокупность результатов сборки, 339 автоматизированных тестов, повторных регрессионных прогонов и ручных сценариев с двумя клиентами подтверждает работоспособность программного комплекса и корректность реализованных механизмов резервного копирования, общего доступа, версионирования, восстановления и обработки временных сетевых сбоев.

ЗАКЛЮЧЕНИЕ

В ходе курсового проекта разработан программный комплекс резервного копирования и синхронизации файлов в клиент-серверной архитектуре. Решение включает локальное клиентское приложение, *SyncGateway*, сервис метаданных, файловое хранилище и *PostgreSQL*. Разделение компонентов позволило независимо организовать пользовательский интерфейс, сетевое взаимодействие, хранение логического состояния и сохранение бинарных версий файлов.

Клиент автоматически отслеживает создание, изменение, переименование и удаление файлов, формирует последовательность неизменяемых версий и позволяет восстанавливать ранее сохранённые состояния. Передача содержимого выполняется бинарным потоком, а размер и *SHA*-256 используются для проверки изменения и целостности. Удаление файла не уничтожает историю, а восстановленная редакция фиксируется как новая текущая версия.

Система поддерживает личные и общие наборы резервного копирования. Участники общего набора используют разные локальные каталоги и постоянные *ClientId*, но работают с единым серверным состоянием и общей историей. Изменения автоматически распространяются между рабочими станциями. При конкурентном редактировании расходящееся локальное содержимое сохраняется как конфликтная копия, что предотвращает потерю одной из пользовательских редакций.

При временной недоступности *SyncGateway* сетевые операции завершаются за ограниченное время, а повторное подключение выполняется в фоновом режиме без блокирования веб-интерфейса. Локальные изменения сохраняются на рабочей станции. После восстановления связи автоматически выполняются новое рукопожатие и согласование, в ходе которого накопленные изменения передаются на сервер и распространяются другим участникам.

Работоспособность комплекса подтверждена модульными, интеграционными, сквозными и регрессионными тестами, а также ручными сценариями с двумя клиентами. Финальная сборка завершилась без ошибок, полный набор из 339 тестов был пройден полностью: 339 успешных проверок, без пропусков и сбоев. Ручные испытания подтвердили двустороннюю синхронизацию, версионирование, восстановление, обработку конфликтов и автоматическое восстановление после сетевого отказа. В результате получена законченная клиент-серверная система, соответствующая поставленной задаче и допускающая дальнейшее развитие.

Список использованных источников

1. Swanson, M. Contingency Planning Guide for Federal Information Systems: NIST SP 800-34 Rev. 1 / M. Swanson, P. Bowen, A. W. Phillips, D. Gallup, D. Lynes [Электронный ресурс]. – Режим доступа: https://doi.org/10.6028/NIST.SP.800-34r1. – Дата доступа: 15.10.2026.

2. What's new in .NET 9 [Электронный ресурс] // Microsoft Learn. – Режим доступа: https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-9/overview. – Дата доступа: 15.10.2026.

3. Asynchronous programming with async and await [Электронный ресурс] // Microsoft Learn. – Режим доступа: https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/. – Дата доступа: 15.10.2026.

4. TcpClient Class (System.Net.Sockets) [Электронный ресурс] // Microsoft Learn. – Режим доступа: https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.tcpclient?view=net-9.0. – Дата доступа: 15.10.2026.

5. FileSystemWatcher Class (System.IO) [Электронный ресурс] // Microsoft Learn. – Режим доступа: https://learn.microsoft.com/ru-ru/dotnet/api/system.io.filesystemwatcher?view=net-9.0. – Дата доступа: 15.10.2026.

6. Сериализация и десериализация JSON в .NET [Электронный ресурс] // Microsoft Learn. – Режим доступа: https://learn.microsoft.com/ru-ru/dotnet/standard/serialization/system-text-json/overview. – Дата доступа: 15.10.2026.

7. ASP.NET Core Blazor [Электронный ресурс] // Microsoft Learn. – Режим доступа: https://learn.microsoft.com/en-us/aspnet/core/blazor/?view=aspnetcore-9.0. – Дата доступа: 15.10.2026.

8. PostgreSQL: About [Электронный ресурс] // PostgreSQL Global Development Group. – Режим доступа: https://www.postgresql.org/about/. – Дата доступа: 15.10.2026.

9. Npgsql Entity Framework Core Provider [Электронный ресурс] // Npgsql Documentation. – Режим доступа: https://www.npgsql.org/efcore/. – Дата доступа: 15.10.2026.

10. Visual Studio documentation [Электронный ресурс] // Microsoft Learn. – Режим доступа: https://learn.microsoft.com/en-us/visualstudio/windows/?view=vs-2022. – Дата доступа: 15.10.2026.

ПРИЛОЖЕНИЕ А
(обязательное)

Архитектурная схема взаимодействия компонентов

ПРИЛОЖЕНИЕ Б
(обязательное)

Текст программы

«AsyncPathLock.cs»

```
using System.Collections.Concurrent;

namespace Backup.Client.Web.Services;

/// <summary>
/// Provides per-path asynchronous locking so that operations on the same
/// relative path are serialized without blocking independent paths.
/// </summary>
public class AsyncPathLock
{
    private readonly ConcurrentDictionary<string, SemaphoreSlim> _locks =
new(StringComparer.OrdinalIgnoreCase);

    public async Task<IDisposable> LockAsync(string path, CancellationToken cancellationToken =
default)
    {
        string normalizedKey = path.Trim().Replace('\\', '/').ToLowerInvariant();
        var semaphore = _locks.GetOrAdd(normalizedKey, _ => new SemaphoreSlim(1, 1));
        await semaphore.WaitAsync(cancellationToken);
        return new Releaser(semaphore);
    }

    private sealed class Releaser : IDisposable
    {
        private SemaphoreSlim? _semaphore;

        public Releaser(SemaphoreSlim semaphore)
        {
            _semaphore = semaphore;
        }

        public void Dispose()
        {
            Interlocked.Exchange(ref _semaphore, null)?.Release();
        }
    }
}
```

«BackupGatewayClient.cs»

```
using System.Net.Sockets;
using Backup.Client.Web.Configuration;
using Backup.Shared.Errors;
using Backup.Shared.Messages;
using Backup.Shared.Protocol;
using Backup.Shared.Validation;
using Microsoft.Extensions.Options;
```

```
namespace Backup.Client.Web.Services;

public class BackupGatewayClient : IAsyncDisposable, IDisposable
{
    private readonly string _host;
    private readonly int _port;
    private readonly ProtocolFrameReader _reader;
    private readonly ProtocolFrameWriter _writer;
    private readonly SemaphoreSlim _connectionLock = new(1, 1);
    private TcpClient? _tcpClient;
    private NetworkStream? _stream;

    public BackupGatewayClient(string host, int port)
    {
        _host = host;
        _port = port;
        _reader = new ProtocolFrameReader();
        _writer = new ProtocolFrameWriter();
    }

    public BackupGatewayClient(IOptions<GatewayOptions> options)
        : this(options.Value.Host, options.Value.Port)
    {
    }

    private static readonly TimeSpan[] ReconnectBackoffs =
    [
        TimeSpan.FromMilliseconds(500),
        TimeSpan.FromSeconds(1),
        TimeSpan.FromSeconds(2),
        TimeSpan.FromSeconds(5)
    ];

    private bool _wasConnectedBefore;
    private Guid? _activeClientId;
    private string _clientName = "Backup.Client.Web";

    public Guid? ActiveClientId => _activeClientId;
    public event Action? Reconnected;

    public void SetActiveClientId(Guid? clientId)
    {
        if (clientId.HasValue && clientId != Guid.Empty)
        {
            _activeClientId = clientId;
        }
    }

    public void MarkDisconnected()
    {
        try { _stream?.Dispose(); } catch { }
```

```
    _stream = null;
    try { _tcpClient?.Dispose(); } catch { }
    _tcpClient = null;
}

public bool IsConnected => _tcpClient != null && _tcpClient.Connected && _stream != null;

private static readonly TimeSpan ConnectTimeout = TimeSpan.FromSeconds(2.5);
private DateTime _lastConnectFailureUtc = DateTime.MinValue;
private readonly object _reconnectLock = new();
private Task? _reconnectTask;
private CancellationTokenSource? _reconnectCts;

public bool IsReconnecting
{
    get
    {
        lock (_reconnectLock)
        {
            return _reconnectTask != null && !_reconnectTask.IsCompleted;
        }
    }
}

public int ActiveReconnectLoopCount
{
    get
    {
        lock (_reconnectLock)
        {
            return (_reconnectTask != null && !_reconnectTask.IsCompleted) ? 1 : 0;
        }
    }
}

public void EnsureBackgroundReconnectRunning()
{
    lock (_reconnectLock)
    {
        if (_reconnectTask != null && !_reconnectTask.IsCompleted)
        {
            return;
        }

        _reconnectCts?.Dispose();
        _reconnectCts = new CancellationTokenSource();
        var token = _reconnectCts.Token;
        _reconnectTask = Task.Run(() => RunBackgroundReconnectLoopAsync(token));
    }
}

private async Task RunBackgroundReconnectLoopAsync(CancellationToken cancellationToken)
```

```
        {
            int backoffIndex = 0;
            while (!cancellationToken.IsCancellationRequested)
            {
                var delay = ReconnectBackoffs[Math.Min(backoffIndex, ReconnectBackoffs.Length - 1)];
                try
                {
                    await Task.Delay(delay, cancellationToken);
                }
                catch (OperationCanceledException)
                {
                    break;
                }

                if (cancellationToken.IsCancellationRequested || IsConnected)
                {
                    break;
                }

                bool acquired = false;
                try
                {
                    acquired = await _connectionLock.WaitAsync(TimeSpan.FromMilliseconds(500), cancel-
lationToken);
                    if (!acquired)
                    {
                        continue;
                    }

                    if (IsConnected)
                    {
                        break;
                    }

                    await ConnectSingleAttemptAsync(cancellationToken);
                    _wasConnectedBefore = true;
                    Reconnected?.Invoke();
                    break;
                }
                catch (Exception ex) when (ex is SocketException or IOException or TimeoutException or
ProtocolException or OperationCanceledException)
                {
                    _lastConnectFailureUtc = DateTime.UtcNow;
                    MarkDisconnected();
                    backoffIndex = Math.Min(backoffIndex + 1, ReconnectBackoffs.Length - 1);
                }
                finally
                {
                    if (acquired)
                    {
                        _connectionLock.Release();
                    }
```

```
            }
        }
    }

    private async Task ConnectSingleAttemptAsync(CancellationToken cancellationToken = default)
    {
        MarkDisconnected();

        _tcpClient = new TcpClient();
        using var timeoutCts = new CancellationTokenSource(ConnectTimeout);
        using var linkedCts = CancellationTokenSource.CreateLinkedTokenSource(cancellationToken,
timeoutCts.Token);

        try
        {
            await _tcpClient.ConnectAsync(_host, _port, linkedCts.Token);
        }
        catch (OperationCanceledException) when (timeoutCts.IsCancellationRequested && !cancella-
tionToken.IsCancellationRequested)
        {
            throw new TimeoutException($"Connection to gateway at {_host}:{_port} timed out after
{ConnectTimeout.TotalSeconds:F1}s.");
        }

        _stream = _tcpClient.GetStream();

        if (_activeClientId.HasValue || _wasConnectedBefore)
        {
            var req = new HelloRequest
            {
                ClientName = _clientName,
                Version = "1.0",
                Role = "Client",
                MachineName = Environment.MachineName,
                ClientId = _activeClientId
            };

            await _writer.WriteJsonFrameAsync(_stream, MessageType.Hello, req, cancellationToken:
linkedCts.Token);
            var respHeader = await _reader.ReadHeaderAsync(_stream, linkedCts.Token);
            if (respHeader == null || respHeader.MessageType != MessageType.HelloAck)
            {
                throw new ProtocolException($"Gateway handshake failed: unexpected response
'{respHeader?.MessageType}'.");
            }
            if (respHeader.BodyLength > 0)
            {
                await _reader.ReadJsonBodyAsync<HelloResponse>(_stream, respHeader.BodyLength,
linkedCts.Token);
            }
        }
    }
```

```
        private async Task ConnectInternalAsync(CancellationToken cancellationToken = default)
        {
            await EnsureConnectedAsync(cancellationToken);
        }

        public async Task ConnectAsync(CancellationToken cancellationToken = default)
        {
            await _connectionLock.WaitAsync(cancellationToken);
            try
            {
                await EnsureConnectedAsync(cancellationToken, force: true);
            }
            finally
            {
                _connectionLock.Release();
            }
        }

        public async Task<HelloResponse?> HelloAsync(
            string clientName = "Backup.Client.Web",
            Guid? clientId = null,
            CancellationToken cancellationToken = default)
        {
            _clientName = clientName;
            if (clientId.HasValue && clientId != Guid.Empty)
            {
                _activeClientId = clientId;
            }

            await _connectionLock.WaitAsync(cancellationToken);
            try
            {
                await EnsureConnectedAsync(cancellationToken);

                var req = new HelloRequest
                {
                    ClientName = clientName,
                    Version = "1.0",
                    Role = "Client",
                    MachineName = Environment.MachineName,
                    ClientId = clientId ?? _activeClientId
                };

                var reqHeader = await _writer.WriteJsonFrameAsync(_stream!, MessageType.Hello, req,
cancellationToken: cancellationToken);
                var respHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);

                if (respHeader == null || respHeader.MessageType != MessageType.HelloAck)
                {
                    MarkDisconnected();
                    return null;
```

```
            }

            return await _reader.ReadJsonBodyAsync<HelloResponse>(_stream!, respHeader.BodyLength, cancellationToken);
        }
        catch (Exception)
        {
            MarkDisconnected();
            throw;
        }
        finally
        {
            _connectionLock.Release();
        }
    }

    public async Task<bool> PingAsync(CancellationToken cancellationToken = default)
    {
        try
        {
            using var pingTimeoutCts = new CancellationTokenSource(TimeSpan.FromSeconds(3));
            using var linkedCts = CancellationTokenSource.CreateLinkedTokenSource(cancellationToken, pingTimeoutCts.Token);

            await _connectionLock.WaitAsync(linkedCts.Token);
            try
            {
                await EnsureConnectedAsync(linkedCts.Token, force: true);
                await _writer.WriteControlFrameAsync(_stream!, MessageType.Ping, cancellationToken: linkedCts.Token);
                var respHeader = await _reader.ReadHeaderAsync(_stream!, linkedCts.Token);
                if (respHeader == null)
                {
                    MarkDisconnected();
                    EnsureBackgroundReconnectRunning();
                    return false;
                }

                if (respHeader.BodyLength > 0)
                {
                    await _reader.ReadBodyBytesAsync(_stream!, respHeader.BodyLength, linkedCts.Token);
                }

                return respHeader.MessageType == MessageType.Pong;
            }
            finally
            {
                _connectionLock.Release();
            }
        }
        catch
```

```
            {
                _lastConnectFailureUtc = DateTime.UtcNow;
                MarkDisconnected();
                EnsureBackgroundReconnectRunning();
                return false;
            }
        }

        public async Task<RegisterClientResponse> RegisterClientAsync(
            RegisterClientRequest request,
            CancellationToken cancellationToken = default)
        {
            await _connectionLock.WaitAsync(cancellationToken);
            try
            {
                await EnsureConnectedAsync(cancellationToken);

                var header = await _writer.WriteJsonFrameAsync(_stream!, MessageType.RegisterClient,
request, cancellationToken: cancellationToken);
                var respHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);

                if (respHeader == null)
                {
                    MarkDisconnected();
                    throw new ProtocolException("Remote host closed TCP connection.");
                }

                if (respHeader.MessageType == MessageType.Error)
                {
                    var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, respHead-
er.BodyLength, cancellationToken);
                    return new RegisterClientResponse
                    {
                        Success = false,
                        ErrorMessage = err?.Message ?? "Registration failed."
                    };
                }

                var result = await _reader.ReadJsonBodyAsync<RegisterClientResponse>(_stream!,
respHeader.BodyLength, cancellationToken);
                if (result != null && result.Success && result.ClientId != Guid.Empty)
                {
                    _activeClientId = result.ClientId;
                }
                return result ?? new RegisterClientResponse { Success = false, ErrorMessage = "Empty Reg-
isterClient response." };
            }
            catch (Exception)
            {
                MarkDisconnected();
                throw;
            }
```

```
            finally
            {
                _connectionLock.Release();
            }
        }

        public async Task<CreateBackupSetResponse> CreateBackupSetAsync(
            CreateBackupSetRequest request,
            CancellationToken cancellationToken = default)
        {
            await _connectionLock.WaitAsync(cancellationToken);
            try
            {
                await EnsureConnectedAsync(cancellationToken);

                var header = await _writer.WriteJsonFrameAsync(_stream!, MessageType.CreateBackupSet,
request, cancellationToken: cancellationToken);
                var respHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);

                if (respHeader == null)
                {
                    MarkDisconnected();
                    throw new ProtocolException("Remote host closed TCP connection.");
                }

                if (respHeader.MessageType == MessageType.Error)
                {
                    var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, respHead-
er.BodyLength, cancellationToken);
                    return new CreateBackupSetResponse
                    {
                        Success = false,
                        ErrorMessage = err?.Message ?? "Failed to create backup set."
                    };
                }

                var result = await _reader.ReadJsonBodyAsync<CreateBackupSetResponse>(_stream!,
respHeader.BodyLength, cancellationToken);
                return result ?? new CreateBackupSetResponse { Success = false, ErrorMessage = "Empty
CreateBackupSet response." };
            }
            catch (Exception)
            {
                MarkDisconnected();
                throw;
            }
            finally
            {
                _connectionLock.Release();
            }
        }
```

```
        public async Task<SyncPlanResponse> SendManifestAsync(
            ManifestRequest request,
            CancellationToken cancellationToken = default)
        {
            await _connectionLock.WaitAsync(cancellationToken);
            try
            {
                await EnsureConnectedAsync(cancellationToken);

                var header = await _writer.WriteJsonFrameAsync(_stream!, MessageType.Manifest, request,
cancellationToken: cancellationToken);
                var respHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);

                if (respHeader == null)
                {
                    MarkDisconnected();
                    throw new ProtocolException("Remote host closed TCP connection.");
                }

                if (respHeader.MessageType == MessageType.Error)
                {
                    var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, respHead-
er.BodyLength, cancellationToken);
                    return new SyncPlanResponse
                    {
                        Success = false,
                        ErrorMessage = err?.Message ?? "Manifest failed."
                    };
                }

                if (respHeader.MessageType != MessageType.SyncPlan)
                {
                    throw new ProtocolException($"Expected SyncPlan, but received '{respHead-
er.MessageType}'.");
                }

                var result = await _reader.ReadJsonBodyAsync<SyncPlanResponse>(_stream!, respHead-
er.BodyLength, cancellationToken);
                return result ?? new SyncPlanResponse { Success = false, ErrorMessage = "Empty SyncPlan
response." };
            }
            catch (Exception)
            {
                MarkDisconnected();
                throw;
            }
            finally
            {
                _connectionLock.Release();
            }
        }
```

```
        public async Task<FileDeletedResponse> DeleteFileAsync(
            FileDeletedRequest request,
            CancellationToken cancellationToken = default)
        {
            await _connectionLock.WaitAsync(cancellationToken);
            try
            {
                await EnsureConnectedAsync(cancellationToken);

                var header = await _writer.WriteJsonFrameAsync(_stream!, MessageType.FileDeleted, request, cancellationToken: cancellationToken);
                var respHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);

                if (respHeader == null)
                {
                    MarkDisconnected();
                    throw new ProtocolException("Remote host closed TCP connection.");
                }

                if (respHeader.MessageType == MessageType.Error)
                {
                    var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, respHeader.BodyLength, cancellationToken);
                    return new FileDeletedResponse
                    {
                        Success = false,
                        ErrorMessage = err?.Message ?? "File deletion failed."
                    };
                }

                if (respHeader.MessageType != MessageType.FileDeleted)
                {
                    throw new ProtocolException($"Expected FileDeleted response, but received '{respHeader.MessageType}'.");
                }

                var result = await _reader.ReadJsonBodyAsync<FileDeletedResponse>(_stream!, respHeader.BodyLength, cancellationToken);
                return result ?? new FileDeletedResponse { Success = false, ErrorMessage = "Empty FileDeleted response." };
            }
            catch (Exception)
            {
                MarkDisconnected();
                throw;
            }
            finally
            {
                _connectionLock.Release();
            }
        }
```

```
        public async Task<FileRenamedResponse> RenameFileAsync(
            FileRenamedRequest request,
            CancellationToken cancellationToken = default)
        {
            await _connectionLock.WaitAsync(cancellationToken);
            try
            {
                await EnsureConnectedAsync(cancellationToken);

                var header = await _writer.WriteJsonFrameAsync(_stream!, MessageType.FileRenamed, request, cancellationToken: cancellationToken);
                var respHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);

                if (respHeader == null)
                {
                    MarkDisconnected();
                    throw new ProtocolException("Remote host closed TCP connection.");
                }

                if (respHeader.MessageType == MessageType.Error)
                {
                    var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, respHeader.BodyLength, cancellationToken);
                    return new FileRenamedResponse
                    {
                        Success = false,
                        ErrorMessage = err?.Message ?? "File rename failed."
                    };
                }

                if (respHeader.MessageType != MessageType.FileRenamed)
                {
                    throw new ProtocolException($"Expected FileRenamed response, but received '{respHeader.MessageType}'.");
                }

                var result = await _reader.ReadJsonBodyAsync<FileRenamedResponse>(_stream!, respHeader.BodyLength, cancellationToken);
                return result ?? new FileRenamedResponse { Success = false, ErrorMessage = "Empty FileRenamed response." };
            }
            catch (Exception)
            {
                MarkDisconnected();
                throw;
            }
            finally
            {
                _connectionLock.Release();
            }
        }
```

```
        /// <summary>
        /// Executes the full E2E backup workflow for a single local file:
        /// 1. Computes streaming hash and length (or uses precomputed).
        /// 2. Sends FileChanged frame to Gateway.
        /// 3. If UploadRequired=false, completes immediately.
        /// 4. If UploadRequired=true, opens FileStream and sends UploadFile binary frame.
        /// 5. Awaits and returns UploadCompletedResponse.
        /// </summary>
        public async Task<UploadCompletedResponse> BackupFileAsync(
            Guid backupSetId,
            string normalizedRelativePath,
            string absoluteFilePath,
            Guid? clientId = null,
            CancellationToken cancellationToken = default,
            string? precomputedSha256 = null,
            long? precomputedSize = null,
            BackupChangeOrigin changeOrigin = BackupChangeOrigin.Normal,
            string? conflictOriginalPath = null,
            int? conflictOriginalVersion = null)
        {
            if (!File.Exists(absoluteFilePath))
            {
                return new UploadCompletedResponse
                {
                    Success = false,
                    ErrorCode = BackupErrorCodes.InvalidLocalFile,
                    ErrorMessage = $"Local file '{absoluteFilePath}' not found."
                };
            }

            await _connectionLock.WaitAsync(cancellationToken);
            try
            {
                await EnsureConnectedAsync(cancellationToken);

                // 1. Calculate length and streaming SHA-256
                long fileLength = precomputedSize ?? FileStreamingHelper.GetFileLength(absoluteFilePath);
                string sha256 = !string.IsNullOrWhiteSpace(precomputedSha256)
                    ? precomputedSha256
                    : await FileStreamingHelper.ComputeFileSha256Async(absoluteFilePath, cancellationToken);

                var changeReq = new FileChangeRequest
                {
                    BackupSetId = backupSetId,
                    ClientId = clientId,
                    RelativePath = normalizedRelativePath,
                    Size = fileLength,
                    Sha256 = sha256,
                    ChangeOrigin = changeOrigin,
                    ConflictOriginalPath = conflictOriginalPath,
```

```
                ConflictOriginalVersion = conflictOriginalVersion
            };

            // 2. Send FileChanged
            var fileChangedHeader = await _writer.WriteJsonFrameAsync(
                _stream!,
                MessageType.FileChanged,
                changeReq,
                cancellationToken: cancellationToken);

            // 3. Receive Gateway response (UploadRequired or UploadCompleted)
            var respHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);
            if (respHeader == null)
            {
                MarkDisconnected();
                throw new ProtocolException("Remote host closed TCP stream while awaiting response to
FileChanged.");
            }

            if (respHeader.MessageType == MessageType.Error)
            {
                var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, respHead-
er.BodyLength, cancellationToken);
                return new UploadCompletedResponse
                {
                    Success = false,
                    ErrorCode = err?.Code ?? "ERROR",
                    ErrorMessage = err?.Message ?? "Gateway returned error."
                };
            }

            if (respHeader.MessageType == MessageType.UploadCompleted)
            {
                // Already up-to-date! No upload required.
                var completedResp = await
_reader.ReadJsonBodyAsync<UploadCompletedResponse>(_stream!, respHeader.BodyLength, cancella-
tionToken);
                return completedResp ?? new UploadCompletedResponse { Success = true };
            }

            if (respHeader.MessageType != MessageType.UploadRequired)
            {
                throw new ProtocolException($"Expected UploadRequired or UploadCompleted, but re-
ceived '{respHeader.MessageType}'.");
            }

            var uploadReq = await _reader.ReadJsonBodyAsync<UploadRequiredResponse>(_stream!,
respHeader.BodyLength, cancellationToken);
            if (uploadReq == null || !uploadReq.UploadRequired)
            {
                return new UploadCompletedResponse
                {
```

```
                Success = true,
                FileId = uploadReq?.FileId,
                VersionNumber = uploadReq?.VersionNumber
            };
        }

        // 4. Verify file hasn't changed since initial hash
        long currentLength = FileStreamingHelper.GetFileLength(absoluteFilePath);
        if (currentLength != fileLength)
        {
            return new UploadCompletedResponse
            {
                Success = false,
                ErrorCode = BackupErrorCodes.FileChangedDuringPrepare,
                ErrorMessage = "Local file size changed between Prepare and Upload phases."
            };
        }

        // 5. Send UploadFile binary frame
        var uploadHeader = new ProtocolHeader
        {
            Magic = ProtocolConstants.Magic,
            ProtocolVersion = ProtocolConstants.Version,
            MessageType = MessageType.UploadFile,
            MessageId = Guid.NewGuid(),
            CorrelationId = fileChangedHeader.MessageId,
            ContentType = ProtocolContentTypes.ApplicationOctetStream,
            BodyLength = fileLength,
            SentAtUtc = DateTime.UtcNow
        };

        await using (var fileStream = new FileStream(
            absoluteFilePath,
            FileMode.Open,
            FileAccess.Read,
            FileShare.Read,
            65536,
            FileOptions.Asynchronous))
        {
            await _writer.WriteFrameAsync(_stream!, uploadHeader, fileStream, cancellationToken);
        }

        // 6. Await UploadCompleted frame
        var completedHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);
        if (completedHeader == null)
        {
            MarkDisconnected();
            throw new ProtocolException("Remote host closed TCP stream while awaiting Upload-
Completed.");
        }

        if (completedHeader.MessageType == MessageType.Error)
```

```
                {
                    var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, completedHeader.BodyLength, cancellationToken);
                    return new UploadCompletedResponse
                    {
                        Success = false,
                        ErrorCode = err?.Code ?? "ERROR",
                        ErrorMessage = err?.Message ?? "Server returned error during commit."
                    };
                }

                if (completedHeader.MessageType != MessageType.UploadCompleted)
                {
                    throw new ProtocolException($"Expected UploadCompleted, but received '{completedHeader.MessageType}'.");
                }

                var finalResult = await _reader.ReadJsonBodyAsync<UploadCompletedResponse>(_stream!, completedHeader.BodyLength, cancellationToken);
                return finalResult ?? new UploadCompletedResponse
                {
                    Success = false,
                    ErrorMessage = "Empty UploadCompleted response received."
                };
            }
            catch (Exception)
            {
                MarkDisconnected();
                throw;
            }
            finally
            {
                _connectionLock.Release();
            }
        }

        public async Task<GetFilesResponse> GetFilesAsync(
            GetFilesRequest request,
            CancellationToken cancellationToken = default)
        {
            await _connectionLock.WaitAsync(cancellationToken);
            try
            {
                await EnsureConnectedAsync(cancellationToken);

                await _writer.WriteJsonFrameAsync(_stream!, MessageType.GetFiles, request, cancellationToken: cancellationToken);
                var respHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);

                if (respHeader == null)
                {
                    MarkDisconnected();
```

```
                throw new ProtocolException("Remote host closed TCP connection.");
            }

            if (respHeader.MessageType == MessageType.Error)
            {
                var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, respHeader.BodyLength, cancellationToken);
                return new GetFilesResponse { Success = false, ErrorMessage = err?.Message ?? "Failed to get files." };
            }

            if (respHeader.MessageType != MessageType.FilesResponse)
            {
                throw new ProtocolException($"Expected FilesResponse, received '{respHeader.MessageType}'.");
            }

            var result = await _reader.ReadJsonBodyAsync<GetFilesResponse>(_stream!, respHeader.BodyLength, cancellationToken);
            return result ?? new GetFilesResponse { Success = false, ErrorMessage = "Empty FilesResponse received." };
        }
        catch (Exception)
        {
            MarkDisconnected();
            throw;
        }
        finally
        {
            _connectionLock.Release();
        }
    }

    public async Task<GetVersionsResponse> GetVersionsAsync(
        GetVersionsRequest request,
        CancellationToken cancellationToken = default)
    {
        await _connectionLock.WaitAsync(cancellationToken);
        try
        {
            await EnsureConnectedAsync(cancellationToken);

            await _writer.WriteJsonFrameAsync(_stream!, MessageType.GetVersions, request, cancellationToken: cancellationToken);
            var respHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);

            if (respHeader == null)
            {
                MarkDisconnected();
                throw new ProtocolException("Remote host closed TCP connection.");
            }
```

```
                if (respHeader.MessageType == MessageType.Error)
                {
                    var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, respHeader.BodyLength, cancellationToken);
                    return new GetVersionsResponse { Success = false, ErrorMessage = err?.Message ?? "Failed to get versions." };
                }

                if (respHeader.MessageType != MessageType.VersionsResponse)
                {
                    throw new ProtocolException($"Expected VersionsResponse, received '{respHeader.MessageType}'.");
                }

                var result = await _reader.ReadJsonBodyAsync<GetVersionsResponse>(_stream!, respHeader.BodyLength, cancellationToken);
                return result ?? new GetVersionsResponse { Success = false, ErrorMessage = "Empty VersionsResponse received." };
            }
            catch (Exception)
            {
                MarkDisconnected();
                throw;
            }
            finally
            {
                _connectionLock.Release();
            }
        }

        public async Task<GetChangeLogResponse> GetChangeLogAsync(
            GetChangeLogRequest request,
            CancellationToken cancellationToken = default)
        {
            await _connectionLock.WaitAsync(cancellationToken);
            try
            {
                await EnsureConnectedAsync(cancellationToken);

                await _writer.WriteJsonFrameAsync(_stream!, MessageType.GetChangeLog, request, cancellationToken: cancellationToken);
                var respHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);

                if (respHeader == null)
                {
                    MarkDisconnected();
                    throw new ProtocolException("Remote host closed TCP connection.");
                }

                if (respHeader.MessageType == MessageType.Error)
                {
```

```
                var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, respHeader.BodyLength, cancellationToken);
                return new GetChangeLogResponse { Success = false, ErrorMessage = err?.Message ?? "Failed to get changelog." };
            }

            if (respHeader.MessageType != MessageType.ChangeLogResponse)
            {
                throw new ProtocolException($"Expected ChangeLogResponse, received '{respHeader.MessageType}'.");
            }

            var result = await _reader.ReadJsonBodyAsync<GetChangeLogResponse>(_stream!, respHeader.BodyLength, cancellationToken);
            return result ?? new GetChangeLogResponse { Success = false, ErrorMessage = "Empty ChangeLogResponse received." };
        }
        catch (Exception)
        {
            MarkDisconnected();
            throw;
        }
        finally
        {
            _connectionLock.Release();
        }
    }

    public async Task<JoinSharedBackupSetResponse> JoinSharedBackupSetAsync(
        JoinSharedBackupSetRequest request,
        CancellationToken cancellationToken = default)
    {
        await _connectionLock.WaitAsync(cancellationToken);
        try
        {
            await EnsureConnectedAsync(cancellationToken);

            await _writer.WriteJsonFrameAsync(_stream!, MessageType.JoinSharedBackupSet, request, cancellationToken: cancellationToken);
            var respHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);

            if (respHeader == null)
            {
                MarkDisconnected();
                throw new ProtocolException("Remote host closed TCP connection.");
            }

            if (respHeader.MessageType == MessageType.Error)
            {
                var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, respHeader.BodyLength, cancellationToken);
```

```
                return new JoinSharedBackupSetResponse { Success = false, ErrorMessage = err?.Message ?? "Failed to join shared backup set." };
            }

            if (respHeader.MessageType != MessageType.JoinSharedBackupSetResult)
            {
                throw new ProtocolException($"Expected JoinSharedBackupSetResult, received '{respHeader.MessageType}'.");
            }

            var result = await _reader.ReadJsonBodyAsync<JoinSharedBackupSetResponse>(_stream!, respHeader.BodyLength, cancellationToken);
            return result ?? new JoinSharedBackupSetResponse { Success = false, ErrorMessage = "Empty JoinSharedBackupSetResult received." };
        }
        catch (Exception)
        {
            MarkDisconnected();
            throw;
        }
        finally
        {
            _connectionLock.Release();
        }
    }

    public async Task<GetBackupSetMembersResponse> GetBackupSetMembersAsync(
        GetBackupSetMembersRequest request,
        CancellationToken cancellationToken = default)
    {
        await _connectionLock.WaitAsync(cancellationToken);
        try
        {
            await EnsureConnectedAsync(cancellationToken);

            await _writer.WriteJsonFrameAsync(_stream!, MessageType.GetBackupSetMembers, request, cancellationToken: cancellationToken);
            var respHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);

            if (respHeader == null)
            {
                MarkDisconnected();
                throw new ProtocolException("Remote host closed TCP connection.");
            }

            if (respHeader.MessageType == MessageType.Error)
            {
                var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, respHeader.BodyLength, cancellationToken);
                return new GetBackupSetMembersResponse { Success = false, ErrorMessage = err?.Message ?? "Failed to get members." };
            }
```

```
                if (respHeader.MessageType != MessageType.GetBackupSetMembersResult)
                {
                    throw new ProtocolException($"Expected GetBackupSetMembersResult, received '{respHeader.MessageType}'.");
                }

                var result = await _reader.ReadJsonBodyAsync<GetBackupSetMembersResponse>(_stream!, respHeader.BodyLength, cancellationToken);
                return result ?? new GetBackupSetMembersResponse { Success = false, ErrorMessage = "Empty GetBackupSetMembersResult received." };
            }
            catch (Exception)
            {
                MarkDisconnected();
                throw;
            }
            finally
            {
                _connectionLock.Release();
            }
        }

        public async Task<GetBackupSetManifestResponse> GetBackupSetManifestAsync(
            GetBackupSetManifestRequest request,
            CancellationToken cancellationToken = default)
        {
            await _connectionLock.WaitAsync(cancellationToken);
            try
            {
                await EnsureConnectedAsync(cancellationToken);

                await _writer.WriteJsonFrameAsync(_stream!, MessageType.GetBackupSetManifest, request, cancellationToken: cancellationToken);
                var respHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);

                if (respHeader == null)
                {
                    MarkDisconnected();
                    throw new ProtocolException("Remote host closed TCP connection.");
                }

                if (respHeader.MessageType == MessageType.Error)
                {
                    var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, respHeader.BodyLength, cancellationToken);
                    return new GetBackupSetManifestResponse { Success = false, ErrorCode = err?.Code ?? "ERROR", ErrorMessage = err?.Message ?? "Failed to get manifest." };
                }

                if (respHeader.MessageType != MessageType.GetBackupSetManifestResult)
                {
```

```
                throw new ProtocolException($"Expected GetBackupSetManifestResult, received '{respHeader.MessageType}'.");
            }

            var result = await _reader.ReadJsonBodyAsync<GetBackupSetManifestResponse>(_stream!, respHeader.BodyLength, cancellationToken);
            return result ?? new GetBackupSetManifestResponse { Success = false, ErrorMessage = "Empty GetBackupSetManifestResult received." };
        }
        catch (Exception)
        {
            MarkDisconnected();
            throw;
        }
        finally
        {
            _connectionLock.Release();
        }
    }

    public async Task<SharedFileDownloadResult> DownloadSharedFileVersionAsync(
        DownloadFileVersionRequest request,
        string localRootPath,
        CancellationToken cancellationToken = default)
    {
        if (string.IsNullOrWhiteSpace(localRootPath) || !Directory.Exists(localRootPath))
        {
            return new SharedFileDownloadResult
            {
                Success = false,
                ErrorCode = "INVALID_ROOT_PATH",
                ErrorMessage = $"Local root path '{localRootPath}' does not exist."
            };
        }

        await _connectionLock.WaitAsync(cancellationToken);
        try
        {
            await EnsureConnectedAsync(cancellationToken);

            // 1. Send DownloadFileVersion request
            await _writer.WriteJsonFrameAsync(_stream!, MessageType.DownloadFileVersion, request, cancellationToken: cancellationToken);

            // 2. Receive DownloadFileVersionReady descriptor
            var readyHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);
            if (readyHeader == null)
            {
                MarkDisconnected();
                throw new ProtocolException("Remote host closed TCP connection before sending DownloadFileVersionReady.");
            }
```

```
            if (readyHeader.MessageType == MessageType.Error)
            {
                var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, readyHeader.BodyLength, cancellationToken);
                return new SharedFileDownloadResult
                {
                    Success = false,
                    ErrorCode = err?.Code ?? "ERROR",
                    ErrorMessage = err?.Message ?? "Download rejected by server."
                };
            }

            if (readyHeader.MessageType != MessageType.DownloadFileVersionReady)
            {
                throw new ProtocolException($"Expected DownloadFileVersionReady, received '{readyHeader.MessageType}'.");
            }

            var ready = await _reader.ReadJsonBodyAsync<DownloadFileVersionReadyResponse>(_stream!, readyHeader.BodyLength, cancellationToken);
            if (ready == null || !ready.Success)
            {
                return new SharedFileDownloadResult
                {
                    Success = false,
                    ErrorCode = ready?.ErrorCode ?? "DOWNLOAD_FAILED",
                    ErrorMessage = ready?.ErrorMessage ?? "DownloadFileVersionReady descriptor indicated failure."
                };
            }

            // 3. Receive DownloadFileVersionData binary header
            var fileHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);
            if (fileHeader == null)
            {
                MarkDisconnected();
                throw new ProtocolException("Remote host closed TCP connection before sending DownloadFileVersionData payload.");
            }

            if (fileHeader.MessageType != MessageType.DownloadFileVersionData)
            {
                throw new ProtocolException($"Expected DownloadFileVersionData binary frame, received '{fileHeader.MessageType}'.");
            }

            if (fileHeader.BodyLength != ready.Size)
            {
                throw new ProtocolException($"Body length mismatch: header has {fileHeader.BodyLength}, ready descriptor has {ready.Size}.");
```

```
            }

            // 4. Validate target path safety
            if (!ProtocolPathValidator.TryNormalizeAndValidate(ready.RelativePath, out var normalizedRelative, out var pathErr))
            {
                await DrainRemainingAsync(_stream!, fileHeader.BodyLength, cancellationToken);
                return new SharedFileDownloadResult
                {
                    Success = false,
                    ErrorCode = BackupErrorCodes.InvalidRelativePath,
                    ErrorMessage = $"Server returned invalid relative path: {pathErr}"
                };
            }

            string fullRoot = Path.GetFullPath(localRootPath).TrimEnd(Path.DirectorySeparatorChar, Path.AltDirectorySeparatorChar) + Path.DirectorySeparatorChar;
            string targetFullPath = Path.GetFullPath(Path.Combine(localRootPath, normalizedRelative));

            if (!targetFullPath.StartsWith(fullRoot, StringComparison.OrdinalIgnoreCase))
            {
                await DrainRemainingAsync(_stream!, fileHeader.BodyLength, cancellationToken);
                return new SharedFileDownloadResult
                {
                    Success = false,
                    ErrorCode = BackupErrorCodes.InvalidRelativePath,
                    ErrorMessage = $"Security violation: Download path '{targetFullPath}' escapes BackupSet root '{fullRoot}'."
                };
            }

            // 5. Ensure parent directories exist
            string? targetDir = Path.GetDirectoryName(targetFullPath);
            if (!string.IsNullOrEmpty(targetDir) && !Directory.Exists(targetDir))
            {
                Directory.CreateDirectory(targetDir);
            }

            // 6. Write to temporary file in the same directory while calculating SHA-256
            string tempPath = Path.Combine(targetDir!, $"{Path.GetFileName(targetFullPath)}.sharedsync.{Guid.NewGuid():N}.tmp");
            string actualSha256;
            long actualSize = 0;

            try
            {
                using (var fileStream = new FileStream(tempPath, FileMode.CreateNew, FileAccess.Write, FileShare.None, 65536, useAsync: true))
                using (var sha256 = System.Security.Cryptography.IncrementalHash.CreateHash(System.Security.Cryptography.HashAlgorithmName.SHA256))
                {
```

```
                byte[] buffer = new byte[65536];
                long remaining = ready.Size;
                while (remaining > 0)
                {
                    int toRead = (int)Math.Min(buffer.Length, remaining);
                    int read = await _stream!.ReadAsync(buffer.AsMemory(0, toRead), cancellationToken);
                    if (read == 0)
                    {
                        throw new ProtocolException($"Unexpected end of stream while streaming download payload. Remaining: {remaining} bytes.");
                    }

                    await fileStream.WriteAsync(buffer.AsMemory(0, read), cancellationToken);
                    sha256.AppendData(buffer, 0, read);
                    actualSize += read;
                    remaining -= read;
                }

                await fileStream.FlushAsync(cancellationToken);
                actualSha256 = Convert.ToHexStringLower(sha256.GetHashAndReset());
            }

            // 7. Validate size and hash
            if (actualSize != ready.Size)
            {
                if (File.Exists(tempPath)) File.Delete(tempPath);
                return new SharedFileDownloadResult
                {
                    Success = false,
                    ErrorCode = BackupErrorCodes.SizeMismatch,
                    ErrorMessage = $"Size mismatch: expected {ready.Size}, received {actualSize}"
                };
            }

            if (!string.Equals(actualSha256, ready.Sha256, StringComparison.OrdinalIgnoreCase))
            {
                if (File.Exists(tempPath)) File.Delete(tempPath);
                return new SharedFileDownloadResult
                {
                    Success = false,
                    ErrorCode = BackupErrorCodes.HashMismatch,
                    ErrorMessage = $"SHA-256 mismatch: expected {ready.Sha256}, calculated {actualSha256}"
                };
            }

            return new SharedFileDownloadResult
            {
                Success = true,
                TempFilePath = tempPath,
                TargetFilePath = targetFullPath,
```

```
                    RelativePath = normalizedRelative,
                    Size = actualSize,
                    Sha256 = actualSha256,
                    VersionNumber = ready.VersionNumber,
                    FileId = ready.FileId
                };
            }
            catch (Exception ex)
            {
                MarkDisconnected();
                if (File.Exists(tempPath))
                {
                    try { File.Delete(tempPath); } catch { }
                }
                return new SharedFileDownloadResult
                {
                    Success = false,
                    ErrorCode = "DOWNLOAD_STREAM_ERROR",
                    ErrorMessage = $"Stream transfer error: {ex.Message}"
                };
            }
        }
        finally
        {
            _connectionLock.Release();
        }
    }

    public async Task<RestoreDownloadResult> DownloadAndVerifyRestoreFileAsync(
        RestoreRequest request,
        string localRootPath,
        CancellationToken cancellationToken = default)
    {
        if (string.IsNullOrWhiteSpace(localRootPath) || !Directory.Exists(localRootPath))
        {
            return new RestoreDownloadResult
            {
                Success = false,
                ErrorCode = "INVALID_ROOT_PATH",
                ErrorMessage = $"Local root path '{localRootPath}' does not exist."
            };
        }

        await _connectionLock.WaitAsync(cancellationToken);
        try
        {
            await EnsureConnectedAsync(cancellationToken);

            // 1. Send RestoreRequest
            await _writer.WriteJsonFrameAsync(_stream!, MessageType.RestoreRequest, request, cancellationToken: cancellationToken);
```

```
                // 2. Receive RestoreReady descriptor
                var readyHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);
                if (readyHeader == null)
                {
                    MarkDisconnected();
                    throw new ProtocolException("Remote host closed TCP connection before sending RestoreReady.");
                }

                if (readyHeader.MessageType == MessageType.Error)
                {
                    var err = await _reader.ReadJsonBodyAsync<ErrorMessage>(_stream!, readyHeader.BodyLength, cancellationToken);
                    return new RestoreDownloadResult
                    {
                        Success = false,
                        ErrorCode = err?.Code ?? "ERROR",
                        ErrorMessage = err?.Message ?? "Restore rejected by server."
                    };
                }

                if (readyHeader.MessageType != MessageType.RestoreReady)
                {
                    throw new ProtocolException($"Expected RestoreReady, received '{readyHeader.MessageType}'.");
                }

                var ready = await _reader.ReadJsonBodyAsync<RestoreReadyResponse>(_stream!, readyHeader.BodyLength, cancellationToken);
                if (ready == null || !ready.Success)
                {
                    return new RestoreDownloadResult
                    {
                        Success = false,
                        ErrorCode = ready?.ErrorCode ?? "RESTORE_FAILED",
                        ErrorMessage = ready?.ErrorMessage ?? "RestoreReady descriptor indicated failure."
                    };
                }

                // 3. Receive RestoreFile binary header
                var fileHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);
                if (fileHeader == null)
                {
                    MarkDisconnected();
                    throw new ProtocolException("Remote host closed TCP connection before sending RestoreFile payload.");
                }

                if (fileHeader.MessageType != MessageType.RestoreFile)
                {
                    throw new ProtocolException($"Expected RestoreFile binary frame, received '{fileHeader.MessageType}'.");
```

```
            }

            if (fileHeader.BodyLength != ready.Size)
            {
                throw new ProtocolException($"Body length mismatch: header has {fileHead-
er.BodyLength}, ready descriptor has {ready.Size}.");
            }

            // 4. Validate target path safety
            if (!ProtocolPathValidator.TryNormalizeAndValidate(ready.RelativePath, out var normalize-
dRelative, out var pathErr))
            {
                await DrainRemainingAsync(_stream!, fileHeader.BodyLength, cancellationToken);
                return new RestoreDownloadResult
                {
                    Success = false,
                    ErrorCode = BackupErrorCodes.InvalidRelativePath,
                    ErrorMessage = $"Server returned invalid relative path: {pathErr}"
                };
            }

            string fullRoot = Path.GetFullPath(localRootPath).TrimEnd(Path.DirectorySeparatorChar,
Path.AltDirectorySeparatorChar) + Path.DirectorySeparatorChar;
            string targetFullPath = Path.GetFullPath(Path.Combine(localRootPath, normalizedRelative));

            if (!targetFullPath.StartsWith(fullRoot, StringComparison.OrdinalIgnoreCase))
            {
                await DrainRemainingAsync(_stream!, fileHeader.BodyLength, cancellationToken);
                return new RestoreDownloadResult
                {
                    Success = false,
                    ErrorCode = BackupErrorCodes.InvalidRelativePath,
                    ErrorMessage = $"Security violation: Restore path '{targetFullPath}' escapes BackupSet
root '{fullRoot}'."
                };
            }

            // 5. Ensure parent directories exist
            string? targetDir = Path.GetDirectoryName(targetFullPath);
            if (!string.IsNullOrEmpty(targetDir) && !Directory.Exists(targetDir))
            {
                Directory.CreateDirectory(targetDir);
            }

            // 6. Write to temporary file in the same directory while calculating SHA-256
            string tempPath = Path.Combine(targetDir!,
$"{Path.GetFileName(targetFullPath)}.restore.{Guid.NewGuid():N}.tmp");
            string actualSha256;
            long actualSize = 0;

            try
            {
```

```
                using (var fileStream = new FileStream(tempPath, FileMode.CreateNew, File-
Access.Write, FileShare.None, 65536, useAsync: true))
                using (var sha256 = Sys-
tem.Security.Cryptography.IncrementalHash.CreateHash(System.Security.Cryptography.HashAlgorithmNa
me.SHA256))
                {
                    byte[] buffer = new byte[65536];
                    long remaining = ready.Size;
                    while (remaining > 0)
                    {
                        int toRead = (int)Math.Min(buffer.Length, remaining);
                        int read = await _stream!.ReadAsync(buffer.AsMemory(0, toRead), cancellationTo-
ken);
                        if (read == 0)
                        {
                            throw new ProtocolException($"Unexpected end of stream while streaming restore
file payload. Remaining: {remaining} bytes.");
                        }

                        await fileStream.WriteAsync(buffer.AsMemory(0, read), cancellationToken);
                        sha256.AppendData(buffer, 0, read);
                        actualSize += read;
                        remaining -= read;
                    }

                    await fileStream.FlushAsync(cancellationToken);
                    actualSha256 = Convert.ToHexStringLower(sha256.GetHashAndReset());
                }

                // 7. Validate size and hash
                if (actualSize != ready.Size)
                {
                    if (File.Exists(tempPath)) File.Delete(tempPath);
                    return new RestoreDownloadResult
                    {
                        Success = false,
                        ErrorCode = "RESTORE_SIZE_MISMATCH",
                        ErrorMessage = $"Size mismatch: expected {ready.Size}, received {actualSize}"
                    };
                }

                if (!string.Equals(actualSha256, ready.Sha256, StringComparison.OrdinalIgnoreCase))
                {
                    if (File.Exists(tempPath)) File.Delete(tempPath);
                    return new RestoreDownloadResult
                    {
                        Success = false,
                        ErrorCode = BackupErrorCodes.HashMismatch,
                        ErrorMessage = $"SHA-256 mismatch: expected {ready.Sha256}, calculated {actu-
alSha256}"
                    };
                }
```

```
                return new RestoreDownloadResult
                {
                    Success = true,
                    TempFilePath = tempPath,
                    TargetFilePath = targetFullPath,
                    RelativePath = normalizedRelative,
                    Size = actualSize,
                    Sha256 = actualSha256,
                    VersionNumber = ready.VersionNumber,
                    FileId = ready.FileId
                };
            }
            catch (Exception ex)
            {
                MarkDisconnected();
                if (File.Exists(tempPath))
                {
                    try { File.Delete(tempPath); } catch { }
                }
                return new RestoreDownloadResult
                {
                    Success = false,
                    ErrorCode = "RESTORE_STREAM_ERROR",
                    ErrorMessage = $"Stream transfer error: {ex.Message}"
                };
            }
        }
        finally
        {
            _connectionLock.Release();
        }
    }

    public async Task<RestoreCompletedResponse> SendRestoreCompletedAsync(
        RestoreCompletedNotification notification,
        CancellationToken cancellationToken = default)
    {
        await _connectionLock.WaitAsync(cancellationToken);
        try
        {
            await EnsureConnectedAsync(cancellationToken);

            await _writer.WriteJsonFrameAsync(_stream!, MessageType.RestoreCompleted, notification, cancellationToken: cancellationToken);
            var respHeader = await _reader.ReadHeaderAsync(_stream!, cancellationToken);

            if (respHeader == null || respHeader.MessageType != MessageType.RestoreCompleted)
            {
                if (respHeader == null) MarkDisconnected();
                return new RestoreCompletedResponse { Success = true };
            }
```

```
            var result = await _reader.ReadJsonBodyAsync<RestoreCompletedResponse>(_stream!, respHeader.BodyLength, cancellationToken);
            return result ?? new RestoreCompletedResponse { Success = true };
        }
        catch (Exception)
        {
            MarkDisconnected();
            throw;
        }
        finally
        {
            _connectionLock.Release();
        }
    }

    private static async Task DrainRemainingAsync(Stream stream, long remainingBytes, CancellationToken cancellationToken)
    {
        if (remainingBytes <= 0) return;
        byte[] buffer = new byte[Math.Min(65536, remainingBytes)];
        long left = remainingBytes;
        while (left > 0)
        {
            int toRead = (int)Math.Min(buffer.Length, left);
            int read = await stream.ReadAsync(buffer.AsMemory(0, toRead), cancellationToken);
            if (read == 0) break;
            left -= read;
        }
    }

    private async Task EnsureConnectedAsync(CancellationToken cancellationToken, bool force = false)
    {
        if (IsConnected && _stream != null)
        {
            return;
        }

        if (!force && DateTime.UtcNow - _lastConnectFailureUtc < TimeSpan.FromSeconds(2.0))
        {
            EnsureBackgroundReconnectRunning();
            throw new SocketException((int)SocketError.ConnectionRefused);
        }

        bool wasConnected = _wasConnectedBefore;
        try
        {
            await ConnectSingleAttemptAsync(cancellationToken);
            _wasConnectedBefore = true;
            if (wasConnected)
            {
```

```
            Reconnected?.Invoke();
        }
    }
    catch (Exception)
    {
        _lastConnectFailureUtc = DateTime.UtcNow;
        MarkDisconnected();
        EnsureBackgroundReconnectRunning();
        throw;
    }
}

public void Dispose()
{
    MarkDisconnected();
    lock (_reconnectLock)
    {
        _reconnectCts?.Cancel();
        _reconnectCts?.Dispose();
        _reconnectCts = null;
    }
    _connectionLock.Dispose();
}

public async ValueTask DisposeAsync()
{
    MarkDisconnected();
    Task? taskToWait = null;
    lock (_reconnectLock)
    {
        _reconnectCts?.Cancel();
        taskToWait = _reconnectTask;
    }
    if (taskToWait != null)
    {
        try { await taskToWait.WaitAsync(TimeSpan.FromSeconds(2)); } catch { }
    }
    lock (_reconnectLock)
    {
        _reconnectCts?.Dispose();
        _reconnectCts = null;
        _reconnectTask = null;
    }
    _connectionLock.Dispose();
}
}
```

ПРИЛОЖЕНИЕ В
(справочное)

Результаты автоматизированного тестирования

Для контроля корректности разработанного программного комплекса выполнено автоматизированное тестирование всего решения *BackupSystem*. Тестовый набор охватывает общие сетевые компоненты, клиентское приложение, сервис метаданных и файлового хранилища, а также интеграционные и сквозные сценарии взаимодействия нескольких клиентских рабочих станций.

Автоматизированные проверки реализованы с использованием *xUnit* и запускаются для всего решения командой *dotnet test BackupSystem.sln --no-build*. Перед финальным прогоном решение было успешно собрано командой *dotnet build BackupSystem.sln* без ошибок и предупреждений.

В составе автоматизированного тестирования проверяются кадрирование *Protocol v*1, сериализация сообщений и полное чтение тела *TCP*-ответа, версионирование и идемпотентная фиксация версии, потоковая передача файлов, вычисление *SHA*-256, работа временных *.part*-файлов и публикация завершённых версий.

Отдельные группы тестов проверяют работу общего хранилища: роли владельца и участника, код подключения *ShareCode*, серверную проверку доступа, единое пространство хранения, автоматическую двустороннюю синхронизацию операций создания, изменения, переименования и удаления, восстановление исторической версии и сохранение конфликтной копии.

Регрессионные и отказоустойчивые проверки охватывают запуск двух независимых клиентов с разными *ClientDataRoot*, сохранение режима *Shared* после перезапуска, отзывчивость *Blazor*-интерфейса при недоступном *SyncGateway*, ограниченные таймауты сетевых операций, отсутствие дублирующих циклов переподключения и автоматическое догоняющее согласование после восстановления связи.

Итоговые показатели финального автоматизированного прогона приведены в таблице В.1.

Таблица В.1 – Итоговые результаты автоматизированного тестирования

Показатель	**Значение**
Всего тестов	339
Успешно выполнено	339
Сбоев	0
Пропущено	0
Продолжительность прогона	27,2 с

Полученный результат показывает, что весь сформированный набор автоматизированных проверок завершился успешно. Все 339 тестов были выполнены, при этом не зафиксировано ни одного сбоя и ни одной пропущенной проверки.

Консольный результат финального запуска тестов представлен на рисунке В.1.

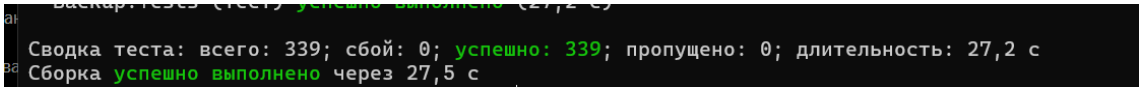


Рисунок В.1 – Результат выполнения автоматизированных тестов

Дополнительно проверялась воспроизводимость регрессионного набора. Исправленный тест страницы истории версий был выполнен двадцать раз подряд без сбоев, после чего два последовательных полных запуска решения также завершились результатом 339 из 339 успешно пройденных тестов. Полученные результаты подтверждают стабильность автоматизированной проверки и соответствие финальной версии программного комплекса реализованным требованиям.

ПРИЛОЖЕНИЕ Г
(справочное)

Демонстрация работы приложения

В приложении представлены основные пользовательские сценарии работы разработанной системы резервного копирования. Приведённые изображения демонстрируют подключение нескольких рабочих станций к общему набору, просмотр истории версий, ведение журнала изменений, разрешение конфликтной ситуации и поведение клиентского интерфейса при временной недоступности сетевого шлюза.

После регистрации клиент может создать общий набор либо подключиться к существующему набору по коду доступа. В интерфейсе отображаются режим набора, роль текущего клиента, количество участников, состояние автоматической синхронизации, интервал фоновой проверки и локальный каталог. Пример состояния общего набора на рабочей станции участника показан на рисунке Г.1.

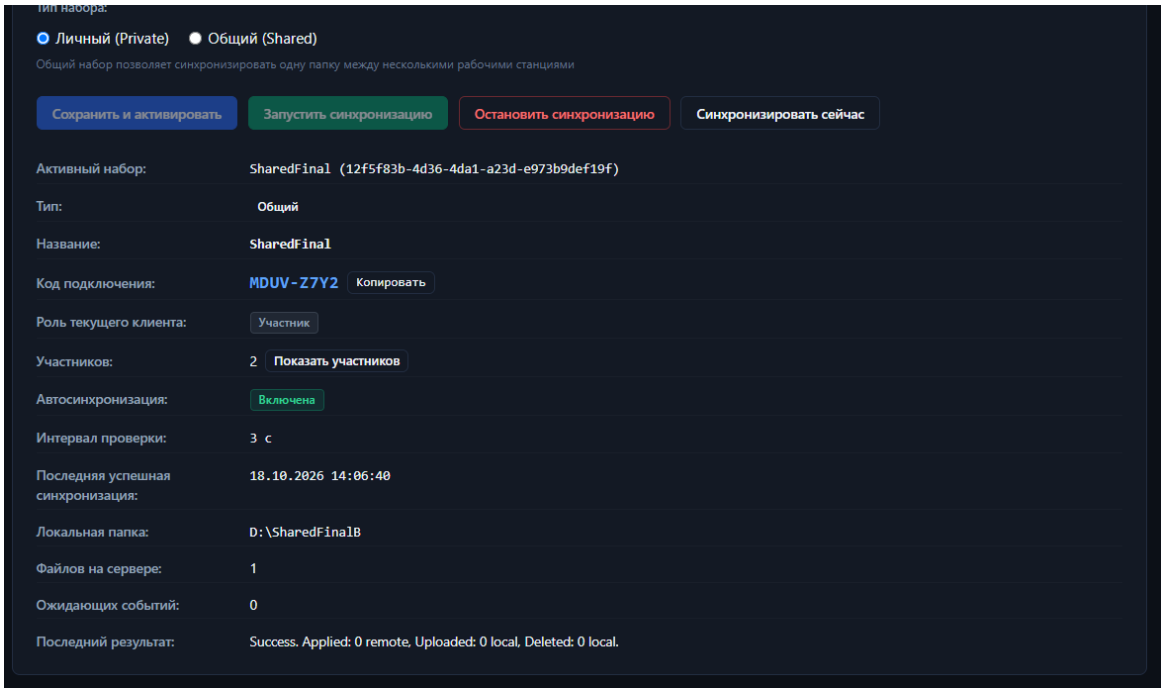


Рисунок Г.1 – Состояние общего набора на рабочей станции участника

Для каждого логического файла пользователь может открыть страницу истории версий. На ней отображаются текущая версия, размер, контрольная сумма *SHA*-256 и ранее сохранённые состояния. Историческую версию можно выбрать для восстановления; восстановленное содержимое формирует новую текущую версию и не удаляет существующую историю. Пример страницы файла приведён на рисунке Г.2.

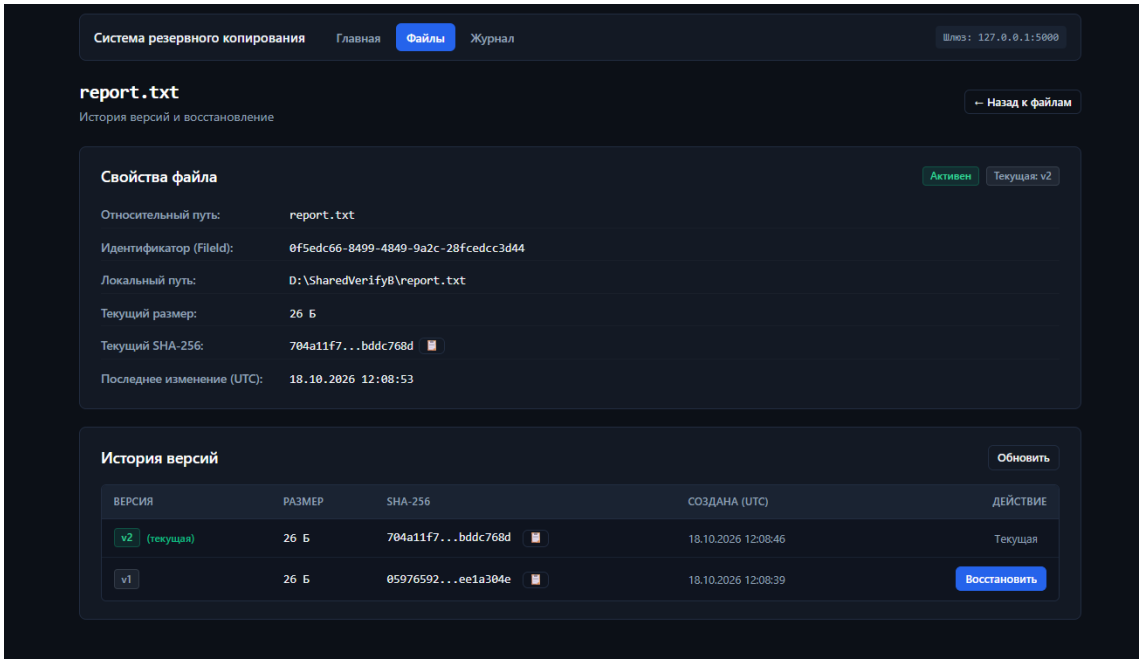


Рисунок Г.2 – История версий файла и восстановление

Журнал изменений общего набора позволяет проследить действия разных рабочих станций. Для каждой операции отображаются дата и время, клиент-инициатор, тип действия, имя файла, состояние и дополнительные сведения. Благодаря этому можно различать создание, изменение, переименование, удаление, восстановление и другие действия участников. Журнал общего набора показан на рисунке Г.3.

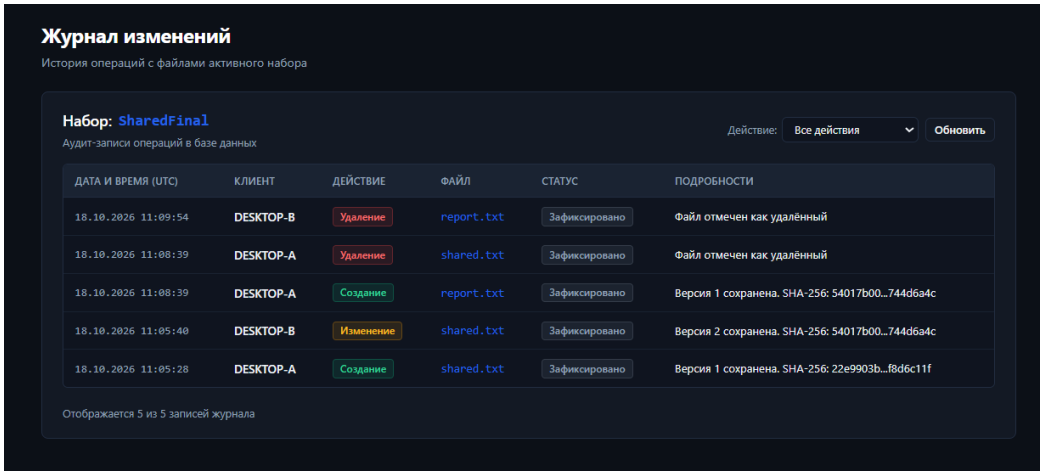


Рисунок Г.3 – Журнал изменений общего набора

При конкурентном изменении одного файла система не выполняет автоматическое слияние и не перезаписывает расходящуюся локальную редакцию. Если клиент обнаруживает, что его изменение основано на устаревшей серверной версии, локальное содержимое сохраняется как самостоятельная кон-

фликтная копия, а основной файл приводится к актуальному серверному состоянию. Наличие основного файла и сохранённой конфликтной копии показано на рисунке Г.4.

```
PS D:\учеба\4 курс\1 сем\coursework_psp> Get-Content "D:\SharedVerifyA\report.txt"
Конфликтная версия от клиента A
PS D:\учеба\4 курс\1 сем\coursework_psp> Set-Content "D:\SharedVerifyB\report.txt" "Основная версия от клиента B" -Encoding UTF8
PS D:\учеба\4 курс\1 сем\coursework_psp> Start-Sleep -Seconds 7
PS D:\учеба\4 курс\1 сем\coursework_psp> Get-Content "D:\SharedVerifyB\report.txt"
Основная версия от клиента B
PS D:\учеба\4 курс\1 сем\coursework_psp> Start-Sleep -Seconds 10
PS D:\учеба\4 курс\1 сем\coursework_psp> Get-ChildItem "D:\SharedVerifyA"


    Каталог: D:\SharedVerifyA


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10/18/2026   3:16 PM             62 report (conflict DESKTOP-MSKCLSN 2026-10-18 121715).txt
-a----        10/18/2026   3:17 PM             56 report.txt
```

Рисунок Г.4 – Сохранение конфликтной копии файла

Отдельно в интерфейсе предусмотрено корректное отображение сетевых сбоев. При остановленном *SyncGateway* страницы остаются доступными и выводят понятное сообщение о недоступности шлюза, не блокируя навигацию. После восстановления соединения клиент автоматически выполняет переподключение и согласование общего набора; состояние системы вновь отображается как синхронизированное. Соответствующие состояния интерфейса представлены на рисунке Г.5.

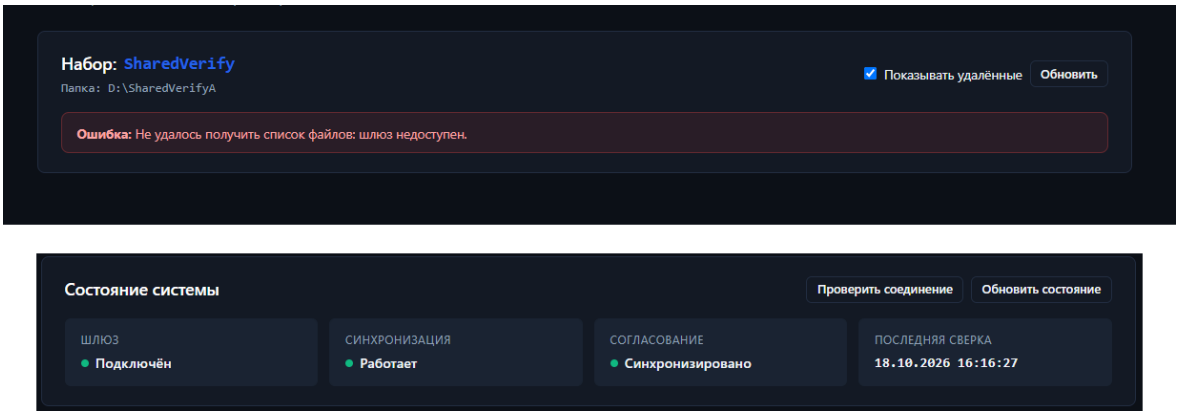


Рисунок Г.5 – Работа клиента при отключении и восстановлении *SyncGateway*

Представленные интерфейсы подтверждают реализацию основных сценариев взаимодействия с программным комплексом. Пользователь может управлять общим набором, контролировать автоматическую синхронизацию, просматривать файлы и историю версий, восстанавливать ранее сохранённые состояния и анализировать журнал операций. При конфликтных изменениях и временных сетевых сбоях система сохраняет пользовательские данные и продолжает работу после восстановления соединения.

ПРИЛОЖЕНИЕ Д
(обязательное)

Руководство пользователя

1 Введение

Система резервного копирования предназначена для автоматического сохранения файлов выбранного локального каталога, ведения истории версий и восстановления ранее сохранённых состояний. Веб-интерфейс позволяет зарегистрировать рабочую станцию, создать личный или общий набор, контролировать синхронизацию, просматривать файлы и журнал изменений.

Специальные навыки программирования пользователю не требуются. В общем наборе каждая рабочая станция использует собственный локальный каталог, а актуальные изменения автоматически распространяются между подключёнными участниками.

2 Назначение и условия применения

Программа используется для резервного копирования папки, хранения последовательных версий, восстановления и контроля операций. В режиме *Shared* несколько рабочих станций подключаются к одному набору по *ShareCode* и автоматически получают изменения других участников.

Необходим современный веб-браузер и запущенные серверные компоненты. При обычном запуске интерфейс клиента доступен по адресу *http*://127.0.0.1:5100, а в демонстрационном режиме интерфейс второго клиента – по адресу *http*://127.0.0.1:5200.

3 Подготовка к работе

Перед работой следует убедиться, что на главной странице шлюз имеет состояние «Подключён». При первом запуске необходимо зарегистрировать клиент; после регистрации приложению назначается постоянный *ClientId*.

Далее указываются название набора и путь к локальной папке. Для личного режима выбирается *Private*. Для совместной работы выбирается *Shared*; владелец получает код подключения, который вводится на второй рабочей станции вместе с путём к её локальной папке.

4 Описание операций

Основные операции выполняются через страницы «Главная», «Файлы» и «Журнал».

– регистрация клиента: нажать «Зарегистрировать клиент» и убедиться, что в блоке текущего клиента появился *ClientId*;

– создание набора: указать название и папку, выбрать *Private* или *Shared* и нажать «Сохранить и активировать»; для *Shared* владельцу выдаётся *ShareCode*;

– подключение участника: на второй рабочей станции ввести *ShareCode* и локальную папку, затем нажать «Подключиться к общему набору»;

– синхронизация: нажать «Запустить синхронизацию». Создание, изменение, переименование и удаление файлов отслеживаются автоматически;

– просмотр файлов и версий: открыть «Файлы», выбрать объект и нажать «Версии» для просмотра истории и восстановления требуемого состояния;

– просмотр журнала: открыть «Журнал». Для общего набора отображается рабочая станция-инициатор и тип выполненного действия;

– остановка: нажать «Остановить синхронизацию». Локальные изменения сохраняются и будут согласованы после следующего запуска.

5 Аварийные ситуации

При недоступности *SyncGateway* интерфейс остаётся работоспособным и сообщает, что шлюз недоступен. После восстановления сервиса клиент автоматически переподключается и выполняет догоняющее согласование. При конкурентном изменении расходящаяся локальная редакция сохраняется как отдельная *conflict-copy*.

Сообщение *ACCESS_DENIED* означает отсутствие доступа текущего *ClientId* к общему набору. Ошибка *ShareCode* указывает на неверный код либо недоступный набор. При длительной недоступности серверных компонентов следует обратиться к системному программисту.

6 Рекомендации по освоению

Для ознакомления рекомендуется создать *Shared*-набор на *Client A*, подключить *Client B* по *ShareCode*, проверить передачу нового файла и обратного изменения, затем выполнить переименование, удаление и восстановление версии и просмотреть журнал операций.

ПРИЛОЖЕНИЕ Е
(обязательное)

Руководство программиста

1 Назначение и условия применения программы

Программный комплекс реализован на *C#* и *.NET* 9 как решение *BackupSystem.sln*. В состав решения входят проекты *Backup.Client.Web*, *Backup.SyncGateway*, *Backup.MetadataService*, *Backup.StorageService*, *Backup.Shared* и *Backup.Tests*. Для хранения метаданных применяется *PostgreSQL* 17, а содержимое версий файлов размещается в отдельном файловом хранилище.

Для разработки требуется *.NET SDK* 9.0.318 либо совместимая версия *.NET* 9. Основной средой разработки может использоваться *Microsoft Visual Studio*. Для полного локального запуска должна быть доступна *PostgreSQL* и настроена строка подключения сервису метаданных.

2 Характеристики программы

Система поддерживает личные и общие наборы резервного копирования, роли *OWNER* и *MEMBER*, подключение по *ShareCode*, автоматическое отслеживание локальных файлов, периодическое согласование общего набора, неизменяемые версии, восстановление, аудит автора операции и сохранение конфликтной копии при конкурентных изменениях.

Сетевое взаимодействие выполняется по собственному *Protocol v*1 поверх *TCP*. Клиенты подключаются только к *SyncGateway*, который маршрутизирует управляющие запросы в *MetadataService* и потоковые операции в *StorageService*. Общие *DTO*, типы сообщений, фрейминг и вспомогательные классы размещены в проекте *Backup.Shared*.

3 Обращение к программе

Сборка решения выполняется командой *dotnet build BackupSystem.sln*. Полный набор тестов запускается командой *dotnet test BackupSystem.sln --no-build*. Финальный прогон содержит 339 тестов и завершается без сбоев и пропусков.

Обычный запуск выполняется .*start.ps*1, остановка — .*stop.ps*1. Для демонстрации двух рабочих станций используются .*start-shared-demo.ps*1 и .*stop-shared-demo.ps*1. В этом режиме *Client A* работает на порту 5100, *Client B* — на порту 5200.

4 Входные и выходные данные

Входными данными являются *ClientId*, параметры профиля синхронизации, *BackupSetId*, *ShareCode*, относительные пути, размеры файлов, *SHA*-256, номера версий и бинарное содержимое. Они поступают от веб-интерфейса и локальной файловой системы.

По сети данные передаются кадрами *Protocol v*1: четырёхбайтовая длина *JSON*-заголовка *UInt*32 *Big-Endian*, *UTF*-8 *JSON*-заголовок и необязательное тело заданной длины. Управляющие сообщения сериализуются в *JSON*, а файлы передаются бинарным потоком без *Base*64.

Выходными данными являются ответы *Protocol v*1, локальный индекс, записи *PostgreSQL* о клиентах, наборах, участниках, файлах, версиях и журнале, а также неизменяемые бинарные версии в файловом хранилище. Локальное состояние сохраняется в *client-state.json*, *sync-profile.json* и файлах индекса *ClientDataRoot*.

5 Сообщения

Прикладные ошибки возвращаются через *ErrorCode* и *ErrorMessage*. *ACCESS_DENIED* означает отсутствие доступа к общему набору. Неверный *ShareCode* формируется при ошибочном формате кода либо отсутствии соответствующего *Shared*-набора. Ошибки размера и *SHA*-256 предотвращают публикацию повреждённой версии.

При недоступности *SyncGateway* клиент ограничивает время *TCP*-попытки и возвращает сообщение «шлюз недоступен». Фоновое переподключение выполняется отдельно от пользовательских запросов; после успешного *reconnect* формируется событие *Reconnected* и запускается внеочередное согласование.

ПРИЛОЖЕНИЕ Ж
(обязательное)

Руководство системного программиста

1 Общие сведения о программе

BackupSystem представляет собой локально развёртываемый клиент-серверный комплекс резервного копирования. В штатной конфигурации работают *Backup.Client.Web*, *Backup.SyncGateway*, *Backup.MetadataService* и *Backup.StorageService*, а метаданные сохраняются в *PostgreSQL* 17.

SyncGateway слушает *TCP*-порт 5000, *MetadataService* — 5001, *StorageService* — 5002. Веб-клиент публикуется на порту 5100; второй экземпляр демонстрационного режима использует 5200. *PostgreSQL* обычно доступна локально на порту 5432.

2 Структура программы

Backup.Client.Web предоставляет интерфейс и работает с локальной папкой. *SyncGateway* обслуживает *TCP*-сессии и координирует внутренние сервисы. *MetadataService* управляет метаданными через *PostgreSQL*, а *StorageService* хранит бинарные версии файлов и потоково выдаёт их при восстановлении.

Состояние обычного клиента размещается в *data*/*client*, а демонстрационных клиентов — в *data*/*client-a* и *data*/*client-b*. Серверное файловое хранилище находится в *data*/*storage*. *PID* запущенных процессов записываются в *data*/*run*/*processes.json* либо *data*/*run*/*shared-demo-processes.json*.

3 Настройка программы

Перед первым запуском необходимо установить *.NET SDK* 9 и *PostgreSQL* 17, подготовить базу *backup_system* и проверить параметры подключения *MetadataService*. *PostgreSQL* должна принимать локальные соединения до запуска *.NET*-сервисов.

Обычный запуск выполняется *.\start.ps1*. Для моделирования двух рабочих станций используется *.\start-shared-demo.ps1*, который задаёт независимые *ClientDataRoot* и *HTTP*-порты. Завершать работу следует соответствующими *stop*-скриптами.

4 Проверка программы

После запуска необходимо открыть *http*://127.0.0.1:5100 и проверить состояние «Шлюз: Подключён». В демонстрационном режиме дополнительно проверяется *http*://127.0.0.1:5200. При необходимости доступность портов 5000, 5001, 5002, 5100 и 5200 проверяется средствами *PowerShell* или *netstat*.

Сборка проверяется командой *dotnet build BackupSystem.sln*, тесты — *dotnet test BackupSystem.sln --no-build*. Эталонный результат финальной версии: сборка без ошибок и предупреждений, 339 успешно выполненных тестов, 0 сбоев и 0 пропущенных.

5 Дополнительные возможности

Для диагностики используются журналы *data*/*logs* и сведения о *PID* в *data*/*run*. Отдельный сервис можно запустить командой *dotnet run --project* с указанием проекта. После изменения исходного кода рекомендуется повторно выполнить сборку и автоматизированные тесты.

Отказоустойчивость можно проверить временной остановкой *SyncGateway*: интерфейс должен оставаться отзывчивым и сообщать о недоступности шлюза, а после повторного запуска клиенты автоматически восстанавливают соединение и выполняют догоняющее согласование.

6 Сообщения системному программисту

Сообщение о занятом адресе означает, что нужный порт уже прослушивается другим процессом. Следует определить его *PID* и корректно остановить предыдущий экземпляр. При ошибке подключения *MetadataService* к *PostgreSQL* проверяются состояние сервера, порт 5432, имя базы и параметры подключения.

Недоступность *MetadataService* или *StorageService* при запуске *SyncGateway* означает, что внутренний сервис не запущен либо не прошёл *Hello*-рукопожатие. Следует проверить соответствующий процесс и журнал. Демонстрационные клиенты должны использовать разные *ClientDataRoot* и независимые *client-state.json* и *sync-profile.json*.

Для штатного завершения рекомендуется использовать *stop.ps*1 или *stop-shared-demo.ps*1, чтобы освободить используемые порты и завершить процессы сервисов в согласованном порядке.